

Specifikacija projekta



1. Osnovne informacije o sistemu

Naziv teme: Laptopi.etf

Nastavna grupa: Grupa 2

Link na repozitorij tima: <https://github.com/ooad-2025-26/Grupa2-Tim3>

Članovi tima:

1. Noa Petrović 19816
2. Rijad Delić 19715
3. Harun Džambas 19310
4. Ali Hebibović 19380

Namjena sistema:

Ovaj sistem je namijenjen za prodaju polovnih i novih laptopa, kao i za iznajmljivanje. Korisnici sistema bit će mlađi i stariji studenti Elektrotehničkog fakulteta. Sistem omogućava studentima viših godina da objave svoju nekorisćenu opremu i prodaju je novijim studentima po povoljnim cijenama. Sistem zahtijeva detaljan opis opreme te povezivanje prodavca i kupca putem slanja e-maila prodavcu. Kupci će moći pregledati sve dostupne artikle. Svaki korisnik će morati da se prijavi i kreira profil kako bi mogao koristiti sistem. Za iznajmljivanje, korisnici će moći vidjeti dostupnost određenog artikla na kalendaru.

2. Funkcionalnosti (poslovni procesi) sistema

Opisati 6 do 8 najznačajnijih funkcionalnosti sistema (u zavisnosti od broja članova u timu). Funkcionalnosti sistema predstavljaju usluge koje sistem pruža korisnicima. Sve funkcionalnosti pripadaju nekoj od različitih vrsta:

- *Usluga sistema - u svrhu ostvarivanja krajnje usluge sistema,*
- *Perzistencija podataka (CRUD operacije)*
- *Asinhrona operacija - operacije koje koriste principe asinhronne obrade zahtjeva*
- *Operacija sa specifičnim algoritmom obrade - operacije koje koriste specifične algoritme obrade podataka,*

- *Korištenje vanjskog uređaja - operacije u kojima se vrši korištenje vanjskih uređaja. Neophodno je navesti barem po jednu funkcionalnost svake od različitih vrsta.*

1) **Naziv funkcionalnosti:** Login/registracija

Vrsta funkcionalnosti: Perzistencija podataka (CRUD operacija)

Opis funkcionalnosti:

Svaki korisnik, kako bi koristio ovaj sistem, mora imati kreiran profil ili da ga napravi. Informacije o pristupnim podacima čuvat će se u bazi podataka. Registracija zahtijeva fakultetski e-mail i šifru.

2) **Naziv funkcionalnosti:** Prodaja artikla

Vrsta funkcionalnosti: Asinhrona operacija

Opis funkcionalnosti:

Prodavač ima mogućnost da postavi svoj artikal na prodaju. Potrebna je detaljna specifikacija, opis, stanje, kao i navođenje mogućih ograničenja koja dolaze uz uređaj. Ukoliko ovi uslovi nisu ispoštovani, administrator ili član tehničke podrške može ukloniti artikal.

3) **Naziv funkcionalnosti:** Iznajmljivanje

Vrsta funkcionalnosti: Asinhrona operacija

Opis funkcionalnosti:

Korisnik ima mogućnost staviti artikal na iznajmljivanje uz detaljan opis i stanje artikla. Usluga iznajmljivanja uključuje kalendar dostupnosti, gdje zainteresovani korisnici mogu vidjeti kada je određen artikal dostupan ili zauzet. Svaki korisnik ima ograničenje korištenja artikla od 15 dana, tek nakon isteka vremenskog perioda od 15 dana ima opciju da opet iznajmi artikal što daje i drugim korisnicima opciju rezervisanja i spriječava rezervisanje na duže periode.

4) **Naziv funkcionalnosti:** Administracija

Vrsta funkcionalnosti: Perzistencija podataka (CRUD operacija)

Opis funkcionalnosti:

Administrator će imati opciju uklanjanja objava za koje se smatra da su neistinite ili da korisnik ne poštuje uslove korištenja. Administrator će putem korisničkog interfejsa imati mogućnost uklanjanja takvih objava.

5) **Naziv funkcionalnosti:** Povezivanje kupca sa prodavačem/iznajmljivačem

Vrsta funkcionalnosti: Korištenje vanjskog uređaja

Opis funkcionalnosti:

Korisnici sistema za prodaju/iznajmljivanje imaju opciju da pritiskom na dugme pošalju vlasniku e-mail sa svojim imenom, prezimenom, brojem telefona i nazivom artikla koji žele kupiti ili iznajmiti.

- 6) **Naziv funkcionalnosti:** Pregled oglasa i sortiranje

Vrsta funkcionalnosti: Operacija sa specifičnim algoritmom obrade

Opis funkcionalnosti:

Korisnici mogu, uz određene filtere, pretraživati artikle koje žele kupiti ili iznajmiti. Filteri omogućavaju pretragu po vremenskom periodu, cijeni ili vrsti usluge. Oglasi će, bez primijenjenih filtera, biti sortirani prema ocjeni korisnika sistema koji je objavio oglas.

- 7) **Naziv funkcionalnosti:** Pregled korisničkog profila

Vrsta funkcionalnosti: Usluga sistema

Opis funkcionalnosti:

Ova usluga omogućava korisniku novi prikaz u kojem može vidjeti određene informacije o drugom korisniku, kao što su ime i prezime, broj postavljenih artikala i broj uspješnih transakcija. Nakon svake transakcije, korisnici će moći ocijeniti druge korisnike, a ta ocjena će biti prikazana na njihovom profilu.

- 8) **Naziv funkcionalnosti:** Tehnička podrška

Vrsta funkcionalnosti: Korištenje vanjskog uređaja

Opis funkcionalnosti:

Korisnici imaju opciju da putem sistema, ukoliko naiđu na problem, pošalju e-mail tehničkoj podršci sa opisom problema.

3. Akteri sistema

Korisnici usluga sistema

- a) **Naziv aktera:** Kupac

Vrsta aktera: Korisnik usluge

Funkcionalnosti u kojima akter učestvuje:

Funkcionalnost sistema	Način učešća
Funkcionalnost 6, pregled oglasa i sortiranje	Mogućnost pregleda
Funkcionalnost 7, pregled korisničkog profila	Mogućnost pregleda
Funkcionalnost 5, povezivanje kupca sa prodavačem/iznajmljivačem	Mogućnost pregleda

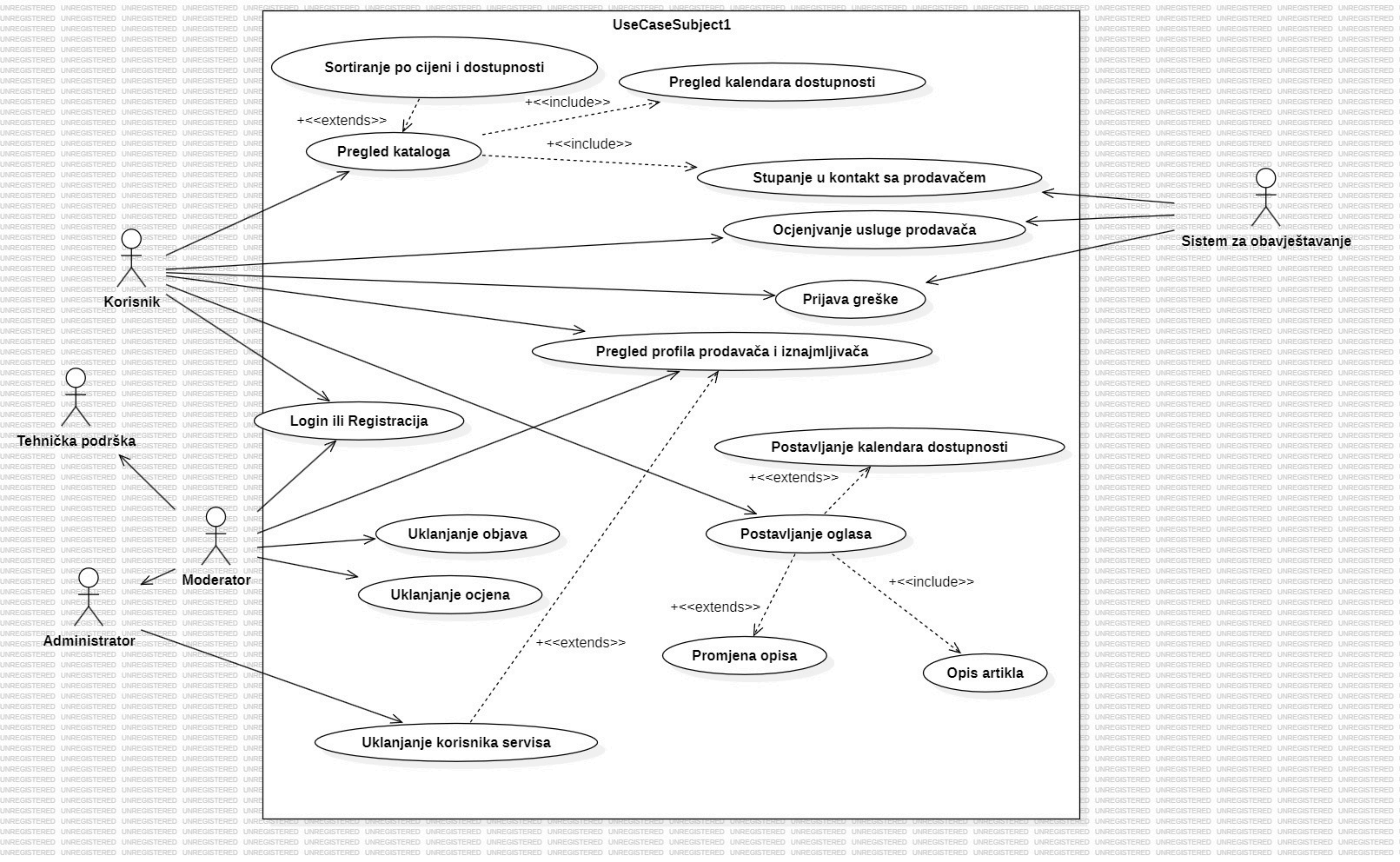
- b) **Naziv aktera: Tehnička podrška** **Vrsta aktera: Zaposlenik sistema**
Funkcionalnosti u kojima akter učestvuje:

Funkcionalnost sistema	Način učešća
Funkcionalnost 8, tehnička podrška	Mogućnost uređivanja
Funkcionalnost 4, administracija	Mogućnost uređivanja
Funkcionalnost 7, pregled korisničkog profila	Mogućnost pregleda

- c) **Naziv aktera: Administrator**
Vrsta aktera: Administrator
Funkcionalnosti u kojima akter učestvuje:

Funkcionalnost sistema	Način učešća
Funkcionalnost 4, administracija	Mogućnost uređivanja
Funkcionalnost 7, pregled korisničkog profila	Mogućnost pregleda

Funkcionalnost 1, login/registracija	Mogućnost uređivanja
--------------------------------------	----------------------



Scenario 1: Login/Registracija

Naziv slučaja upotrebe	Login/Registracija
Opis slučaja upotrebe	Korisnik kreira profil ili se prijavljuje u sistem pomoću fakultetskog e-maila i šifre
Vezani zahtjevi	Validacija e-maila, čuvanje podataka u bazi, kreiranje profila
Preduslovi	Korisnik mora imati fakultetski e-mail
Posljedice – uspješan završetak	Korisnik dobija pristup sistemu
Posljedice – neuspješan završetak	Korisnik ne može pristupiti sistemu
Primarni akteri	Korisnik
Ostali akteri	Administrator (može uređivati podatke)
Glavni tok	Korisnik: Unosi e-mail i šifru-> Sistem: Validira podatke i kreira profil
Alternative/proširenja	Korisnik: Unosi pogrešan e-mail -> Sistem: Odbija registraciju

Tok događaja 1: Uspješna registracija/login

Korisnik	Sistem
Otvora formu za prijavu/registraciju	

Unosi fakultetski e-mail i šifru	
Potvrđuje unos podataka	
	Validira format e-maila i ispravnost podataka
	Provjerava da li korisnik već postoji u bazi
	ako korisnik postoji → vrši prijavu (login) ako korisnik ne postoji → kreira novi profil
	Omogućava pristup sistemu

Tok događaja 2: Ne uspješna registracija/login

Korisnik	Sistem
Otvora formu za prijavu/registaciju	
Unosi e-mail i šifru	
Potvrđuje unos podataka	
	Validira e-mail i ispravnost podataka
	Utvrdjuje da e-mail ili podaci nisu ispravni
	Odbija prijavu/registaciju
	Prikazuje poruku o grešci

Scenario 2: Prodaja artikla

Naziv slučaja upotrebe	Objavljivanje artikla za prodaju
Opis slučaja upotrebe	Prodavač postavlja artikal sa detaljnim opisom i cijenom
Vezani zahtjevi	Asinhrono objavljivanje, validacija unosa, mogućnost uklanjanja od strane administratora
Preduslovi	Korisnik mora biti registrovan i prijavljen

Posljedice – uspješan završetak	Artikal je objavljen i vidljiv kupcima
Posljedice – neuspješan završetak	Artikal nije objavljen
Primarni akteri	Korisnik
Ostali akteri	Administrator
Glavni tok	Korisnik: Unosi opis, stanje i cijenu-> Sistem: Objavljuje artikal
Alternative/proširenja	Korisnik: Ne unosi sve podatke -> Sistem: Traži dopunu

Tok događaja 1: Uspješna prodaja (objavljivanje) artikla

Korisnik	Sistem
Otvora formu za objavu artikla	Prikazuje formu za unos podataka o artiklu
Unosi naziv, opis, stanje i cijenu artikla	
Dodaje slike artikla (opcionalno)	
Potvrđuje unos podataka	Validira e-mail i ispravnost podataka
	Provjerava da li su svi obavezni podaci uneseni
	Sprema podatke u bazu
	Objavljuje artikal i omogućava vidljivost kupcima

Tok događaja 2: Neuspješna prodaja (objavljivanje) artikla

Korisnik	Sistem
Otvora formu za objavu artikla	Prikazuje formu za unos podataka
Unosi nepotpune ili neispravne podatke	
Potvrđuje unos podataka	Validira unesene podatke
	Utvrdjuje da podaci nisu ispravni ili potpuni
	Odbija objavu artikla
	Prikazuje poruku o grešci
	Traži ispravan unos podataka

Scenario 3: Iznajmljivanje artikla

Naziv slučaja upotrebe	Iznajmljivanje artikla
Opis slučaja upotrebe	Prodavač stavlja artikal na iznajmljivanje, kupac rezerviše period
Vezani zahtjevi	Kalendar dostupnosti, ograničenje na 15 dana
Preduslovi	Korisnik mora biti registrovan
Posljedice – uspješan završetak	Artikal rezervisan na odabrani period
Posljedice – neuspješan završetak	Rezervacija odbijena
Primarni akteri	Kupac
Ostali akteri	Prodavač
Glavni tok	Korisnik: Bira period iznajmljivanja-> Sistem: Provjera dostupnost -> Iznajmljivač: Odobrava zahtjev -> Sistem: Šalje potvrdu odobrenog zahtjeva

Alternative/proširenja	Korisnik: Pokuša rezervisati duže od 15 dana -> Sistem: Odbija zahtjev
	Korisnik: Pokus ava rezervisat artikal -> Iznajmljivač: Odbija zahtjev/Ne odgovori u roku od 24 sata -> Sistem: Š alje obavjest odbijenog zahtjeva

Tok događaja 1: Uspješno iznajmljivanje

Korisnik	Sistem	Iznajmljivač
1. Korisnik bira artikal koji z eli iznajmit		
2. Korisnik bira period iznajmljivanja		
	3. Š istem provjerava je li period iznajmljivanja zauzet i je li krac i od 15 dana	
4. Korisnik potvrđuje zahtjev za iznajmljivanje		
	5. Š istem s alje mail iznajmljivac u za potvrdu transakcije	
		6. Iznajmljivac potvrđuje transakciju
	7. Š istem postavlja odabrane datume kao zauzete i s alje potvrdu korisniku o odobrenoj transakciji	8.

Tok događaja 2: Neuspješno iznajmljivanje

Korisnik	Sistem	Iznajmljivač
1. Korisnik bira artikal koji zeli iznajmit		
2. Korisnik bira period iznajmljivanja		
	3. Šistem provjerava je li period iznajmljivanja zauzet i je li krac i od 15 dana	
4. Korisnik potvrđuje zahtjev za iznajmljivanje		
	5. Šistem s alje mail iznajmljivac u za potrvdu transakcije	
		6. Iznajmljivac ne odgovara u roku od 24 sata
	7. Šistem s alje obavijest korisniku da transakcija nije odobrena	

Tok događaja 3: Neuspješno iznajmljivanje

Korisnik	Sistem	Iznajmljivač
1. Korisnik bira artikal koji zeli iznajmit		
2. Korisnik bira period iznajmljivanja		
	3. Šistem provjerava je li period iznajmljivanja zauzet i je li krac i od 15 dana	

4. Korisnik potvrđuje zahtjev za iznajmljivanje		
	5. Š istem s alje mail iznajmljivac u za potvrdu transakcije	
		6. Iznajmljivac odbija transakciju
	7. Š istem s alje obavijest korisniku da transakcija nije odobrena	

Tok događaja 4: Neuspješan odabir perioda iznajmljivanja

Korisnik	Sistem	Iznajmljivač
1. Korisnik bira artikal koji z eli iznajmit		
2. Korisnik bira period iznajmljivanja		
	3. Š istem provjerava je li period iznajmljivanja zauzet i je li krac i od 15 dana	
	4. Š istem utvrđuje da zahtej iznajmljivanja izlazi van dozvoljenog opsega i vrac a korisnika na ponovni odabir perioda iznajmljivanja	

Scenario 4: Povezivanje kupca sa prodavačem

Naziv slučaja upotrebe	Kontaktiranje prodavača
Opis slučaja upotrebe	Kupac šalje e-mail prodavaču putem sistema
Vezani zahtjevi	Generisanje e-maila sa podacima kupcima
Preduslovi	Kupac mora biti prijavljen
Posljedice – uspješan završetak	Prodavač prima e-mail sa podacima kupca
Posljedice – neuspješan završetak	E-mail nije poslan
Primarni akteri	Kupac
Ostali akteri	Prodavač
Glavni tok	Korisnik: Klikne na dugme <i>Kontaktiraj</i> -> Sistem: Generiše i šalje e-mail
Alternative/proširenja	Sistem: Ako e-mail servis nije dostupan, obavještava kupca

Tok događaja 1: Uspješno kontaktiranje prodavača

Korisnik	Sistem
Pregledava artikal	Prikazuje detalje artikla
Klikne na dugme "Kontaktiraj"	
Unosi poruku (opcionarno)	
Potvrđuje slanje	Prikuplja podatke o kupcu (e-mail, ime)
	Generiše e-mail sa podacima kupca i sadržajem poruke
	Šalje e-mail prodavaču
	Obavještava kupca da je poruka uspješno poslana

Tok događaja 2: Neuspješno kontaktiranje prodavača

Korisnik	Sistem
Pregledava artikal	Prikazuje detalje artikla
Klikne na dugme "Kontaktiraj"	
Pokušava poslati poruku	Pokušava generisati i poslati e-mail
	Utvrdjuje da e-mail servis nije dostupan
	Ne uspijeva poslati e-mail
	Prikazuje poruku o grešci
	Obavještava kupca da slanje nije uspjelo

Scenario 5: Pregled i sortiranje oglasa

Naziv slučaja upotrebe	Pregled i sortiranje oglasa
Opis slučaja upotrebe	Kupac pretražuje oglase uz filtere
Vezani zahtjevi	Š ortiranje po ocjeni, filtriranje po cijeni, periodu i vrsti usluge
Preduslovi	Kupac mora biti prijavljen
Posljedice – uspješan završetak	Kupac vidi relevantne oglase
Posljedice – neuspješan završetak	Nema rezultata pretrage
Primarni akteri	Kupac
Ostali akteri	Š istem
Glavni tok	Korisnik: Primjenjuje filtere -> Sistem: Š ortira i prikazuje rezultate
Alternative/proširenja	Sistem: Ako nema rezultata, prikazuje poruku "Nema dostupnih artikala"

Tok događaja 1: Uspješan prikaz artikala nakon sortiranja

Korisnik	Sistem
1. Korisnik bira filtere koje z eli primjenit na pretragu	
	2. Š istem provjerava postoje li artikli koji spadaju u traz enu kategoriju
	3. Š istem prikazuje traz ene artikle

Tok događaja 2: Neuspješan prikaz artikala nakon sortiranja

Korisnik	Sistem
1. Korisnik bira filtere koje z eli primjenit na pretragu	
	2. Š istem provjerava postoje li artikli koji spadaju u traz enu kategoriju
	3. Š istem izbacije poruku da ne postoje artikli koji spadaju u traz enu kategoriju

Scenario 6: Moderacija artikala i korisnika

Naziv slučaja upotrebe	Moderacija artikala i korisnika
Opis slučaja upotrebe	Moderator ručno pregledava artikle i korisničke profile, prima primjedbe od korisnika i u slučaju nepravilnosti šalje zahtjev administratoru za uklanjanje
Vezani zahtjevi	Pregled artikala i profila, prijem korisničkih primjedbi, slanje zahtjeva administratoru
Preduslovi	Moderator mora biti prijavljen u sistem

Posljedice – uspješan završetak	Zahtjev za uklanjanje je poslan administratoru, nepravilni artikal ili korisnik može biti uklonjen
Posljedice – neuspješan završetak	Zahtjev nije poslan artikal ili korisnik ostaje aktivan
Primarni akteri	Moderator (tehnička podrška)
Ostali akteri	Administrator, Korisnici (kupci/prodavači)
Glavni tok	Moderator: Pregledava artikle i korisnike -> Sistem: Prikazuje detalje artikla/profila Moderator: Prima primjedbe od korisnika -> Sistem: Evidentira primjedbu Moderator: Odlučuje da artikal/korisnik
	treba ukloniti -> Sistem: Generiše zahtjev i šalje administratoru
Alternative/proširenja	Moderator: Zaključuje da artikal/korisnik ne krši pravila -> Sistem: Ne šalje zahtjev administratoru Sistem: Ako primjedba nije validna, obavještava moderatora

Tok događaja 1: Slanje zahtjeva administratoru

Moderator	Sistem
Prijavljuje se u sistem	
Pregledava listu artikala i korisničkih profila	
	Prikazuje detalje odabranog artikla ili profila
Prima ili pregleda primjedbe korisnika	
	Evidentira primjedbu u sistemu
Analizira artikal ili profil	
Zaključuje da postoji nepravilnost	
Pokreće zahtjev za uklanjanje	

	Generiše zahtjev za uklanjanje
	Šalje zahtjev administratoru

Tok događaja 2: Nema nepravilnosti

Moderator	Sistem
Prijavljuje se u sistem	
Pregledava listu artikala i korisničkih profila	
	Prikazuje detalje odabranog artikla ili profila
Prima ili pregleda primjedbe korisnika	
	Evidentira primjedbu u sistemu
Analizira artikal ili profil	
Zaključuje da artikal ili korisnik ne krši pravila	
	Ne generiše zahtjev za uklanjanje

Tok događaja 3: Nevalidna primjedba

Moderator	Sistem
Pregledava primjedbu korisnika	
	Evidentira primjedbu u sistemu
Analizira primjedbu	
	Provjerava validnost primjedbe
	Utvrdjuje da primjedba nije validna
	Obavještava moderatora o nevalidnoj primjedbi
Uklanja primjedbu	

Scenario 7: Administracija

Naziv slučaja upotrebe	Administracija
Opis slučaja upotrebe	Administrator uklanja objave koje krše pravila

Vezani zahtjevi	CRUD operacije nad objavama
Preduslovi	Administrator mora biti prijavljen
Posljedice – uspješan završetak	Objave uklonjene, korisnik obaviješten
Posljedice – neuspješan završetak	Objave ostanu aktivne
Primarni akteri	Administrator
Ostali akteri	Tehnička podrška
Glavni tok	Korisnik (Admin): Pregledava objave -> Sistem: Uklanja objavu i obavještava korisnika
Alternative/proširenja	Ako objava nije problematična -> ostaje aktivna

Tok 1: Uspješno uklanjanje objave

Korisnik (Administrator)	Sistem
1. Prijavljuje se u sistem	2. Potvrđuje prijavu i otvara admin preglednik
3. Identifikuje problematičnu objavu	
4. Odabira opciju <i>Ukloni artikal</i>	5. Potvrđuje da je artikal aktivan
	6. Uklanja objavu i obavještava korisnika da je objava uklonjena

Tok 2: Neuspješno uklanjanje objave

Korisnik	Sistem
1. Prijavljuje sistem	2. Potvrđuje prijavu i otvara admin preglednik
3. Identifikuje problematičnu objavu	
4. Odabira opciju <i>Ukloni artikal</i>	5. Potvrđuje da je artikal ipak neaktivan
	6. Obavještava administratora da uklanjanje nije uspjelo

Tok 3: Pregled objave koja nije problematična

Korisnik	Sistem
1. Prijavljuje sistem	2. Potvrđuje prijavu i otvara admin preglednik
3. Identifikuje problematičnu objavu	
4. Zaključuje da objava ne krši pravila platforme	5. Ostavlja objavu aktivnom

Tok 4: Uklanjanje na osnovu prijave moderatora

Korisnik	Sistem
1. Prijavljuje sistem	2. Potvrđuje prijavu i otvara admin preglednik
3. Odabira opciju <i>Obavještenja od moderatora</i>	4. Prikazuje listu prijavljenih objava sa komentarima moderatora
5. Odabira objavu označenu kao problematičnu	6. Omogućava opciju <i>Ukloni objavu</i>
7. Utvrđuje da objava krši pravila platforme i odabire opciju <i>Ukloni artikal</i>	8. Uklanja objavu iz baze i obavještava korisnika i korisnika da je objava uklonjena

Tok 5: Uklanjanje na osnovu prijave moderatora

Korisnik	Sistem
1. Prijavljuje se u sistem	2. Potvrđuje prijavu i otvara admin preglednik
3. Odabira opciju <i>Obavještenja od moderatora</i>	4. Prikazuje listu prijavljenih objava sa komentarima moderatora
5. Odabira objavu označenu kao problematičnu	6. Omogućava opciju <i>Ukloni objavu</i>
7. Utvrđuje da objava ne krši pravila platforme i odbacuje prijavu objave	8. Objava ostaje aktivna

Scenario 8: Dodavanje artikla

Naziv slučaja upotrebe	Dodavanje artikla
Opis slučaja upotrebe	Korisnik dodaje novi artikal (oglas) u sistem sa detaljnim informacijama
Vezani zahtjevi	CRUD operacije, validacija podataka, upload slika
Preduslovi	Korisnik mora biti prijavljen
Posljedice – uspješan završetak	Artikal je uspješno dodan i vidljiv drugim korisnicima
Posljedice – neuspješan završetak	Artikal nije dodan u sistem
Primarni akteri	Korisnik
Ostali akteri	Šistem
Glavni tok	Korisnik: Otvara formu za dodavanje artikla → Korisnik: Unosi naziv, opis, cijenu i ostale podatke → Korisnik: Dodaje slike artikla → Sistem: Validira unesene podatke → Sistem: Sprema artikal u bazu → Sistem: Prikazuje potvrdu o uspješnom dodavanju
Alternative/proširenja	Korisnik: Ne unese sve obavezne podatke → Sistem: Prikazuje grešku i traži dopunu → Sistem: Dođe do greške pri spremanju → Sistem: Obavještava korisnika da artikal nije dodan

Tok 1: Uspješno objavljivanje artikla

Korisnik	Sistem
1. Prijavljuje se u sistem	2. Potvrđuje prijavu i otvara korisnički interfejs
3. Odabira opciju <i>Dodaj artikal</i>	4. Otvara formu za unos podataka
5. Unosi podatke (naziv, opis, cijena, stanje)	6. Validira podatke
7. Potvrđuje podatke	8. Sprema artikal u bazu podataka i prikazuje obavijest o uspješnom dodavanju

Tok 2: Neuspješno objavljivanje artikla

Korisnik	Sistem
1. Prijavljuje se u sistem	2. Potvrđuje prijavu i otvara korisnički interfejs
3. Odabira opciju <i>Dodaj artikal</i>	4. Otvara formu za unos artikla
5. Unosi podatke (naziv, opis, cijena, stanje)	6. Nailazi na grešku (npr. nevalidni podaci, nedostaju obavezna polja)
7. Pokušava da potvrdi dodavanje	8. Obavještava korisnika da artikal nije dodan i traži ispravku podataka

Scenario 9: Uklanjanje artikla

Naziv slučaja upotrebe	Uklanjanje artikla
Opis slučaja upotrebe	Korisnik ili administrator uklanja postojeći artikal iz sistema

Vezani zahtjevi	CRUD operacije, kontrola pristupa
Preduslovi	Korisnik mora biti prijavljen i vlasnik artikla ili administrator
Posljedice – uspješan završetak	Artikal je uklonjen iz sistema
Posljedice – neuspješan završetak	Artikal ostaje u sistemu
Primarni akteri	Korisnik
Ostali akteri	Administrator
Glavni tok	Korisnik: Pronalazi svoj artikal → Korisnik: Odabire opciju za brisanje → Sistem: Traži potvrdu akcije → Korisnik: Potvrđuje brisanje → Sistem: Briše artikal iz baze → Sistem: Ažurira prikaz oglasa
Alternative/proširenja	Korisnik: Odustane od brisanja → Sistem: Zadržava artikal u sistemu → Sistem: Korisnik nema dozvolu → Sistem: Odbija akciju i prikazuje poruku

Tok događaja 1: Uspješno uklanjanje artikla

Korisnik (vlasnik artikla) / Administrator	Sistem
Pronalazi artikal	Prikazuje listu artikala
Odabire opciju za brisanje	
	Traži potvrdu akcije
Potvrđuje brisanje	
	Provjerava da li korisnik ima dozvolu (vlasnik ili administrator)
	Briše artikal iz baze
	Ažurira prikaz oglasa
	Prikazuje poruku o uspješnom brisanju

Tok događaja 2: Neuspješno uklanjanje artikla

Korisnik (vlasnik artikla) / Administrator	Sistem
Pronalazi artikal	Prikazuje listu artikala
Odabire opciju za brisanje	
	Provjerava da li korisnik ima dozvolu (vlasnik ili administrator)
	Utvrdjuje da korisnik nema dozvolu
	Odbija akciju brisanja
	Prikazuje poruku o grešci

Tok događaja 3: Odustajanje od brisanja

Korisnik (vlasnik artikla) / Administrator	Sistem
Pronalazi artikal	Prikazuje listu artikala
Odabire opciju za brisanje	
	Traži potvrdu akcije
Odustaje od brisanja	
	Prekida proces brisanja
	Zadržava artikal u sistemu

Scenario 10: Ocjenjivanje artikla

Naziv slučaja upotrebe	Ocjenjivanje artikla
Opis slučaja upotrebe	Korisnik ocjenjuje artikal nakon pregleda ili kupovine

Vezani zahtjevi	Algoritam za izračun prosječne ocjene, validacija
Preduslovi	Korisnik mora biti prijavljen
Posljedice – uspješan završetak	Ocjena artikla je zabilježena i ažuriran je prosjek
Posljedice – neuspješan završetak	Ocjena nije zabilježena
Primarni akteri	Korisnik
Ostali akteri	Š istem
Glavni tok	Korisnik: Otvara stranicu artikla → Korisnik: Odabire ocjenu (npr. 1–5) → Korisnik: Potvrđuje unos → Sistem: Sprema ocjenu → Sistem: Ažurira prosječnu ocjenu artikla → Sistem: Prikazuje novu ocjenu
Alternative/proširenja	Korisnik: Pokuša ocijeniti više puta → Sistem: Odbija unos i obavještava korisnika → Sistem: Dode do greške → Sistem: Obavještava korisnika da ocjena nije spremljena

Tok događaja 1: Uspješno izvršeno ocjenjivanje artikla

Korisnik	Sistem
Otvora stranicu artikla	
Odabire ocjenu (npr. 1-5)	
Potvrđuje unos ocjene	
	Provjerava da li je korisnik već ocijenio artikal
	Sprema ocjenu u bazu podataka
	Izračunava novu prosječnu ocjenu artikla
	Prikazuje ažuriranu ocjenu artikla

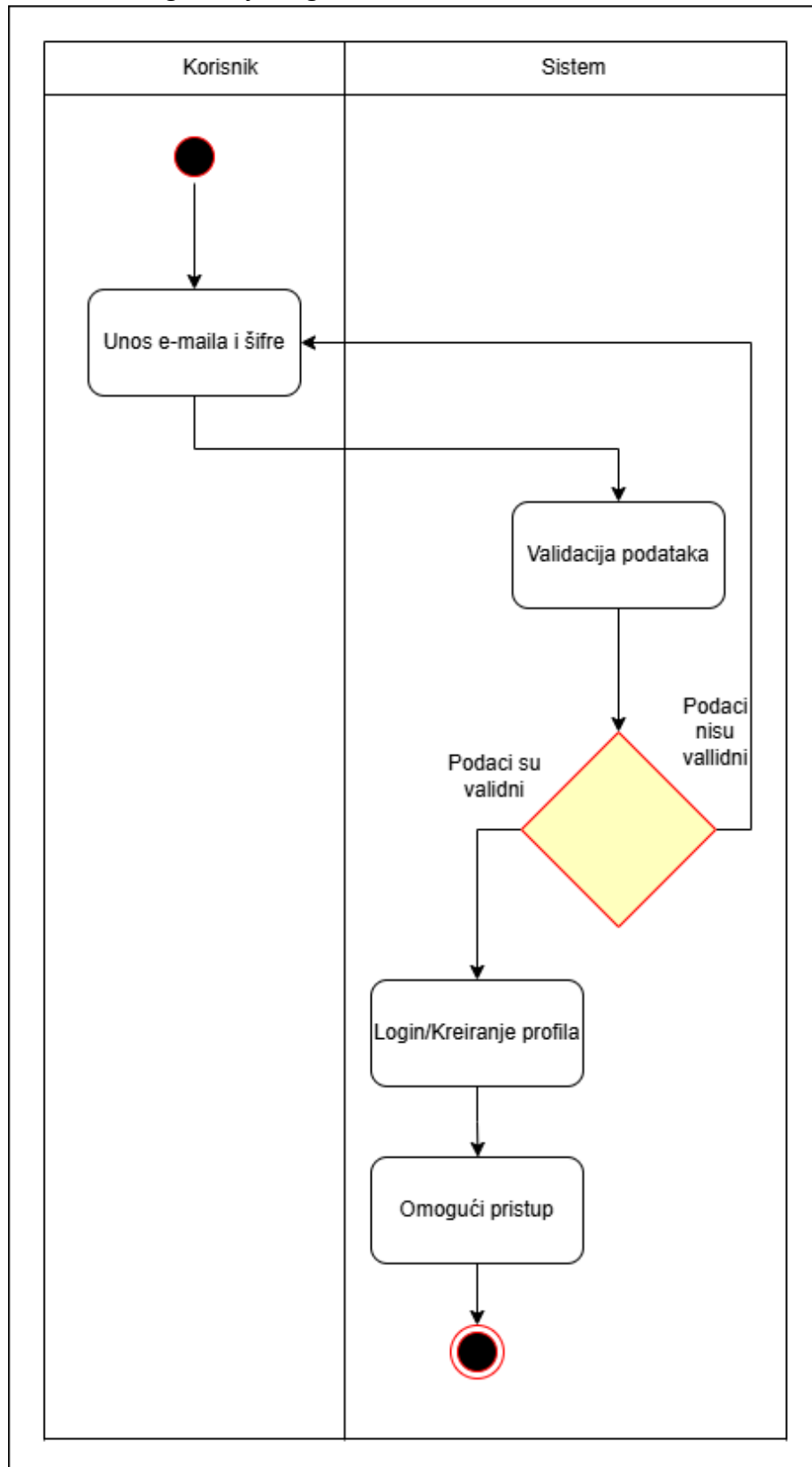
Tok događaja 2: Neuspješan unos ocjene artikla (Korisnik već ocijenio artikal)

Korisnik	Sistem
Otvora stranicu artikla	
Pokušava unijeti ocjenu	
	Provjerava da li je korisnik već ocijenio artikal
	Utvrđuje da je artikal već ocijenjen
	Prikazuje poruku da ponovno ocjenjivanje nije dozvoljeno

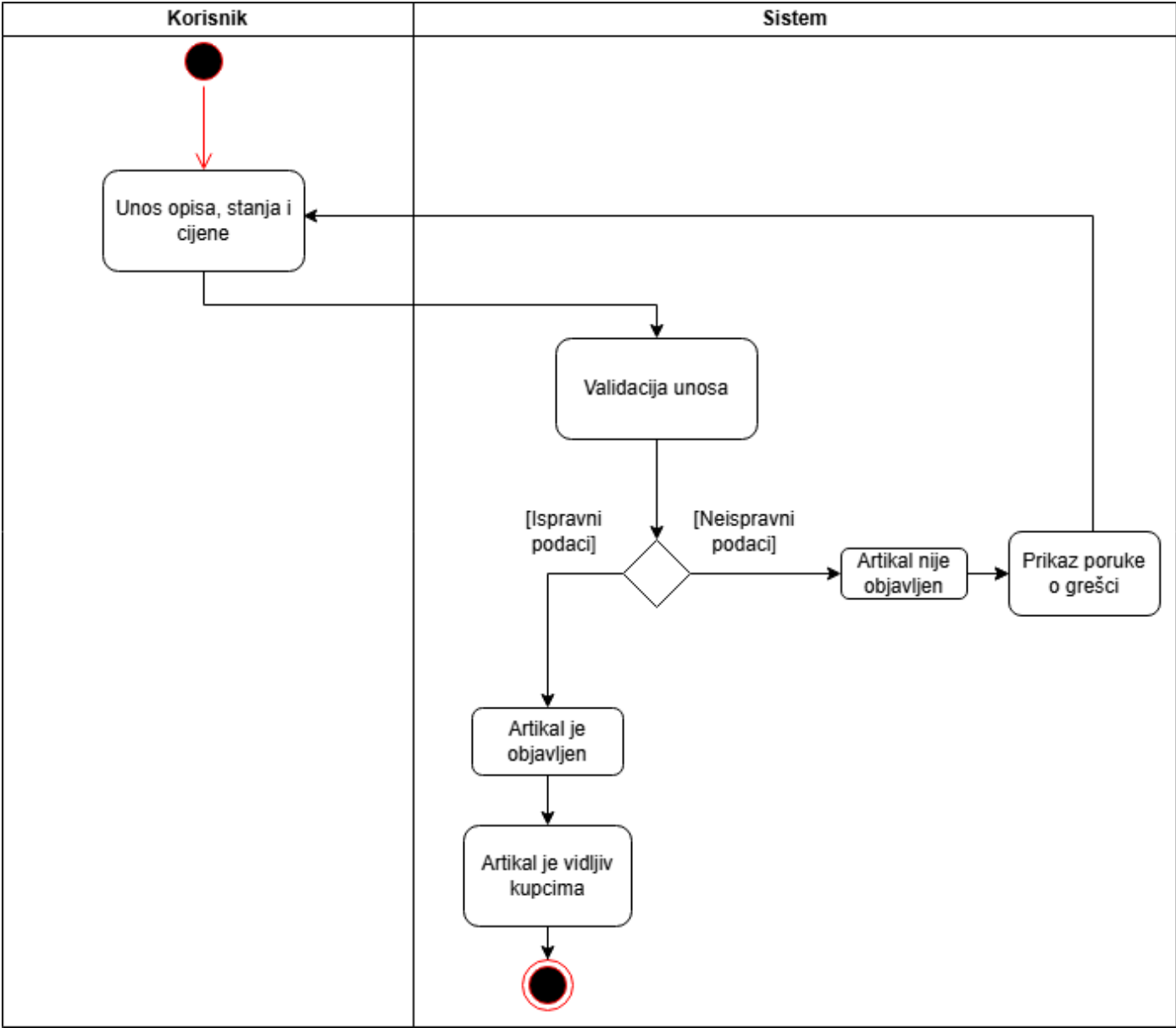
Tok događaja 3: Neuspješan unos ocjene (Greška pri spremanju ocjene)

Korisnik	Sistem
Otvora stranicu artikla	
Unosi ocjenu	
Potvrđuje unos ocjene	
	Pokušava spremiti ocjenu u bazu
	Dolazi do greške pri spremanju
	Obavještava korisnika da ocjena nije zabilježena

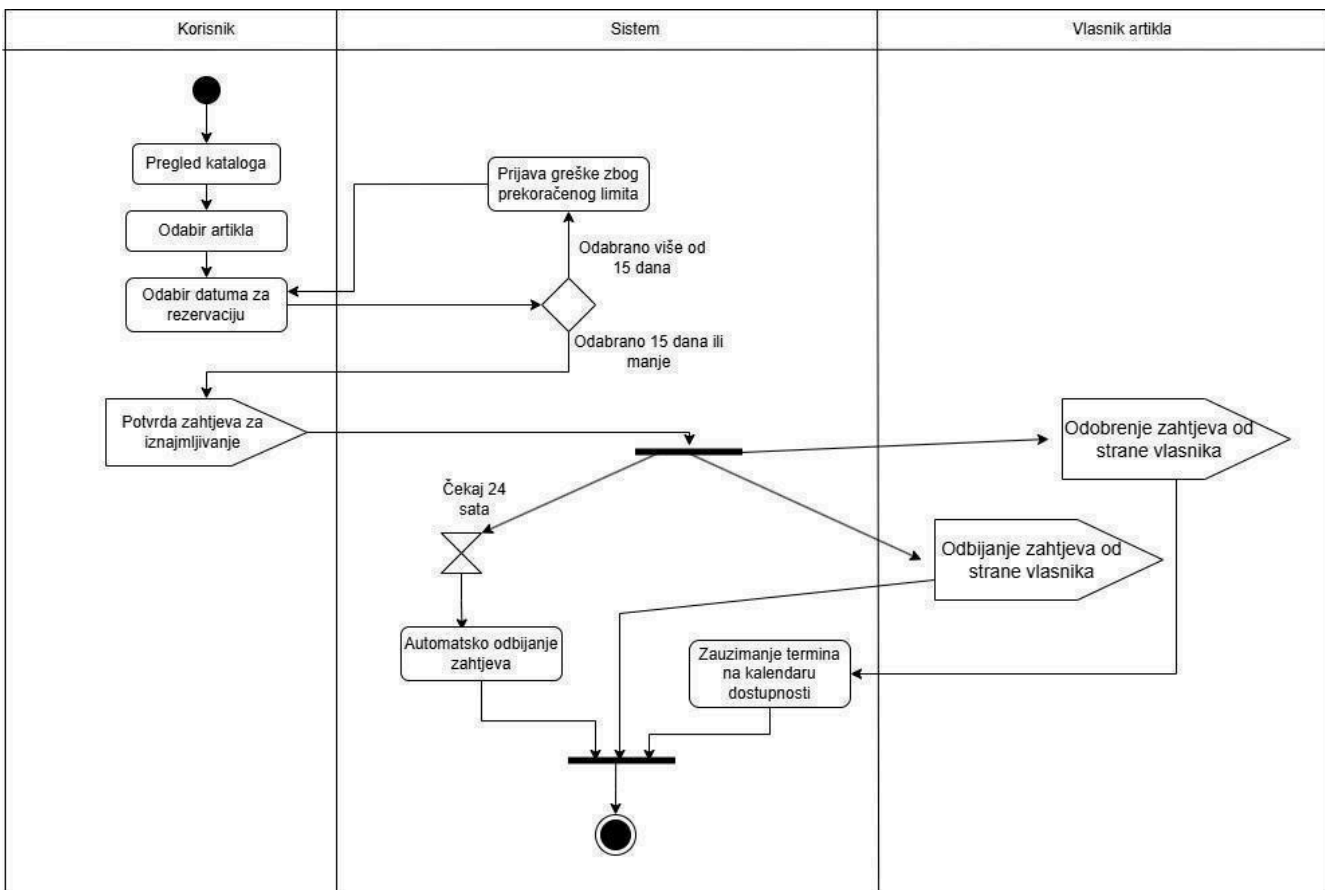
Scenario 1: Registracija/Login



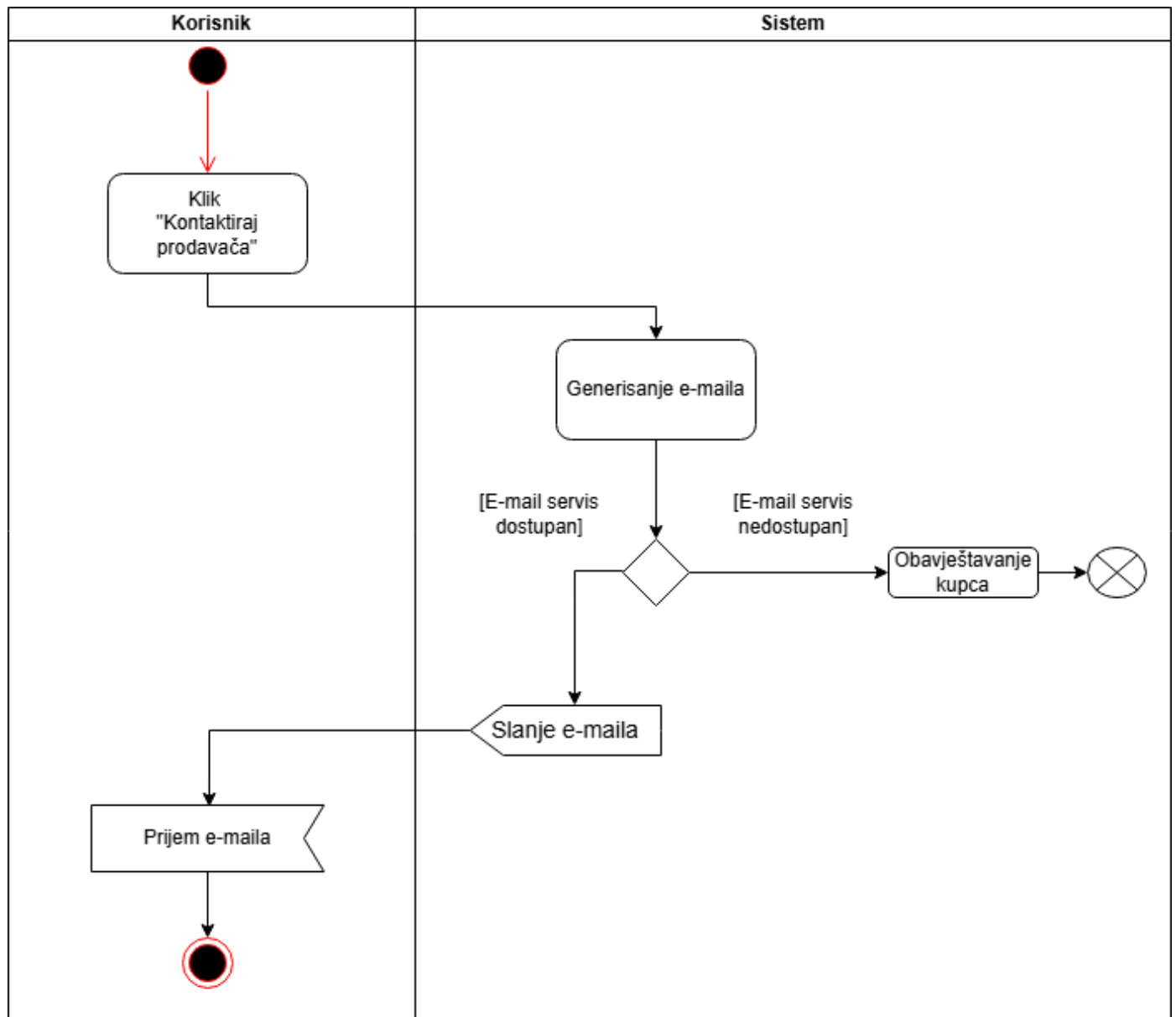
Scenario 2: Prodaja artikla



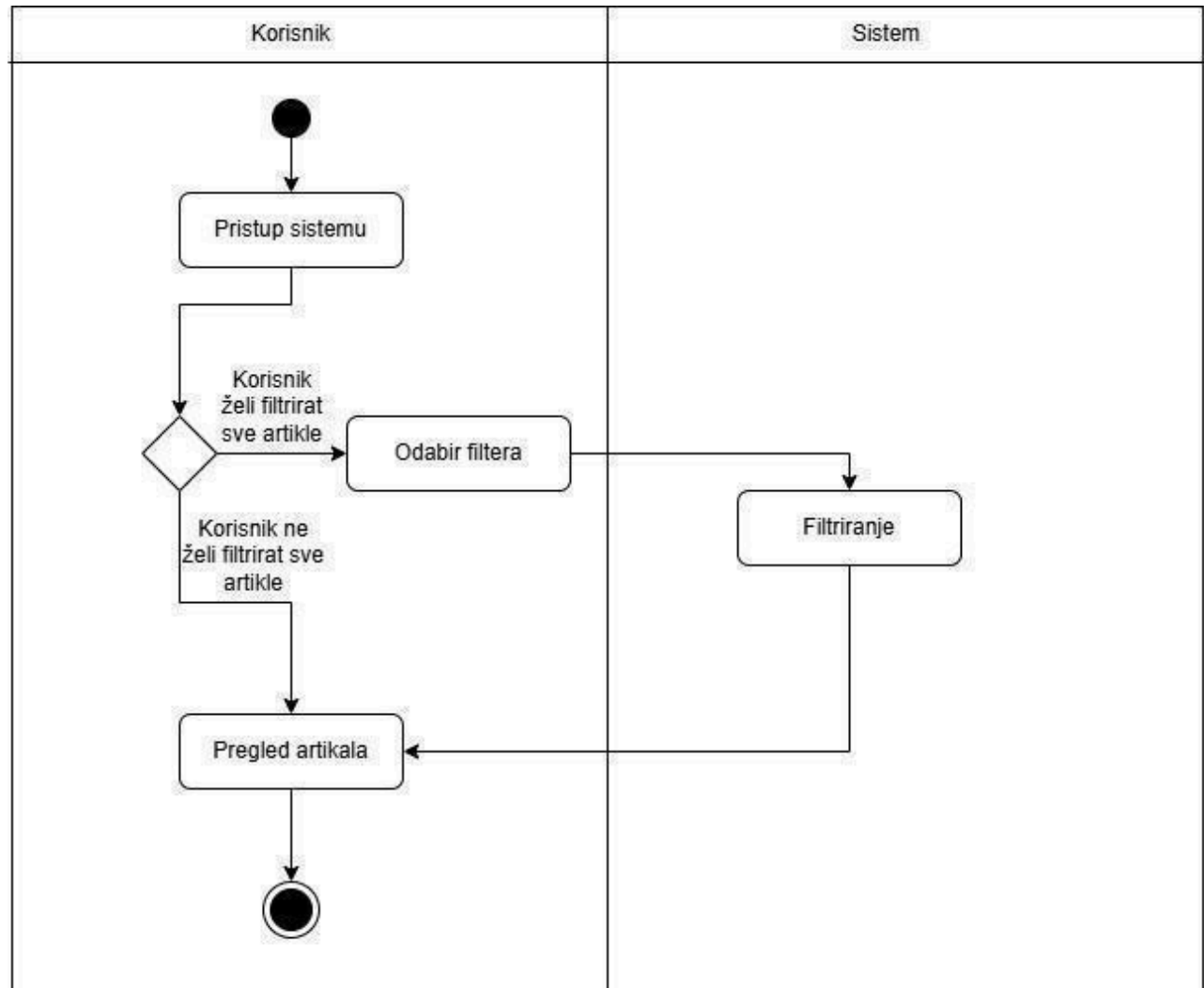
Scenario 3: Iznajmljivanje artikla



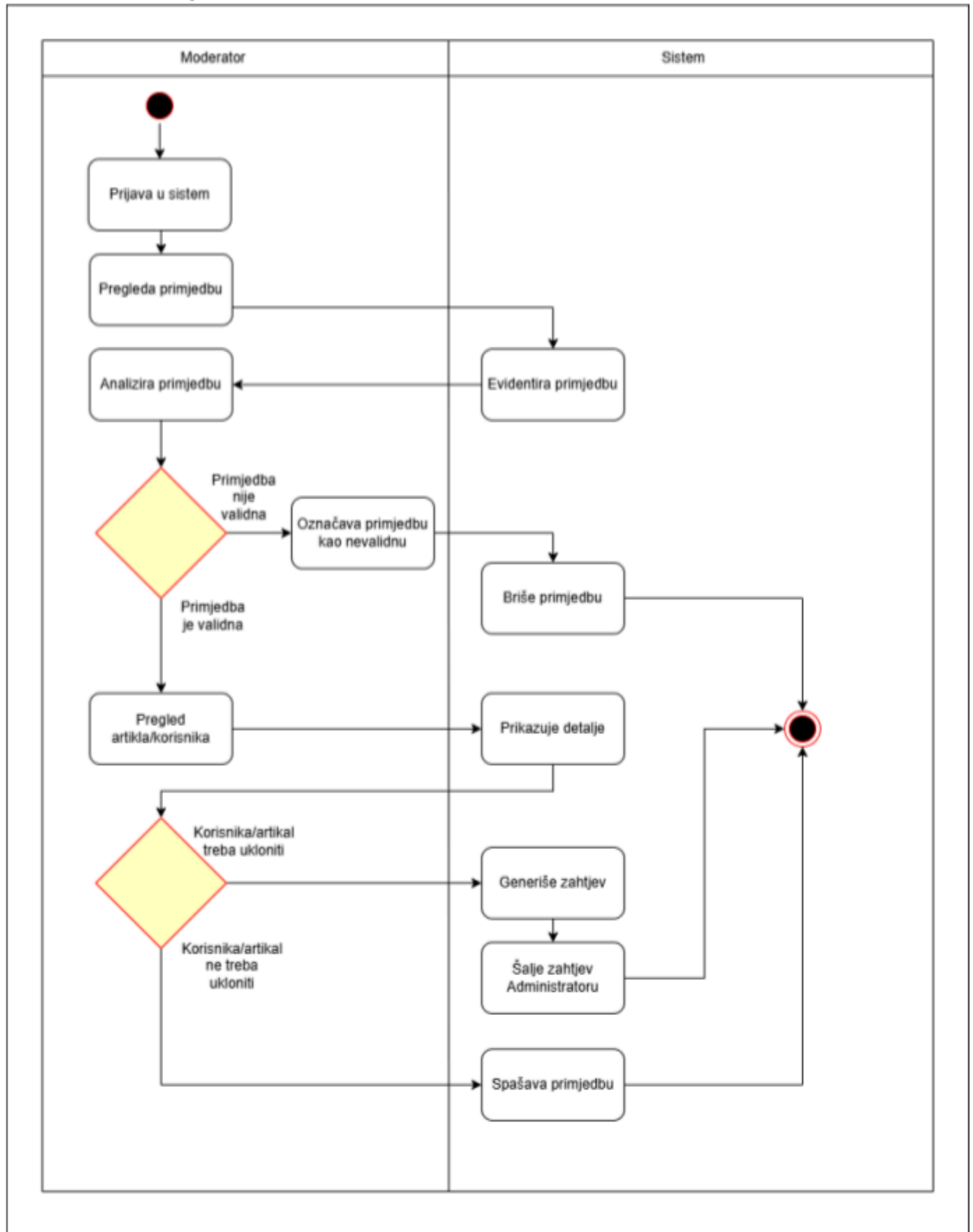
Scenario 4: Povezivanje kupca sa prodavačem



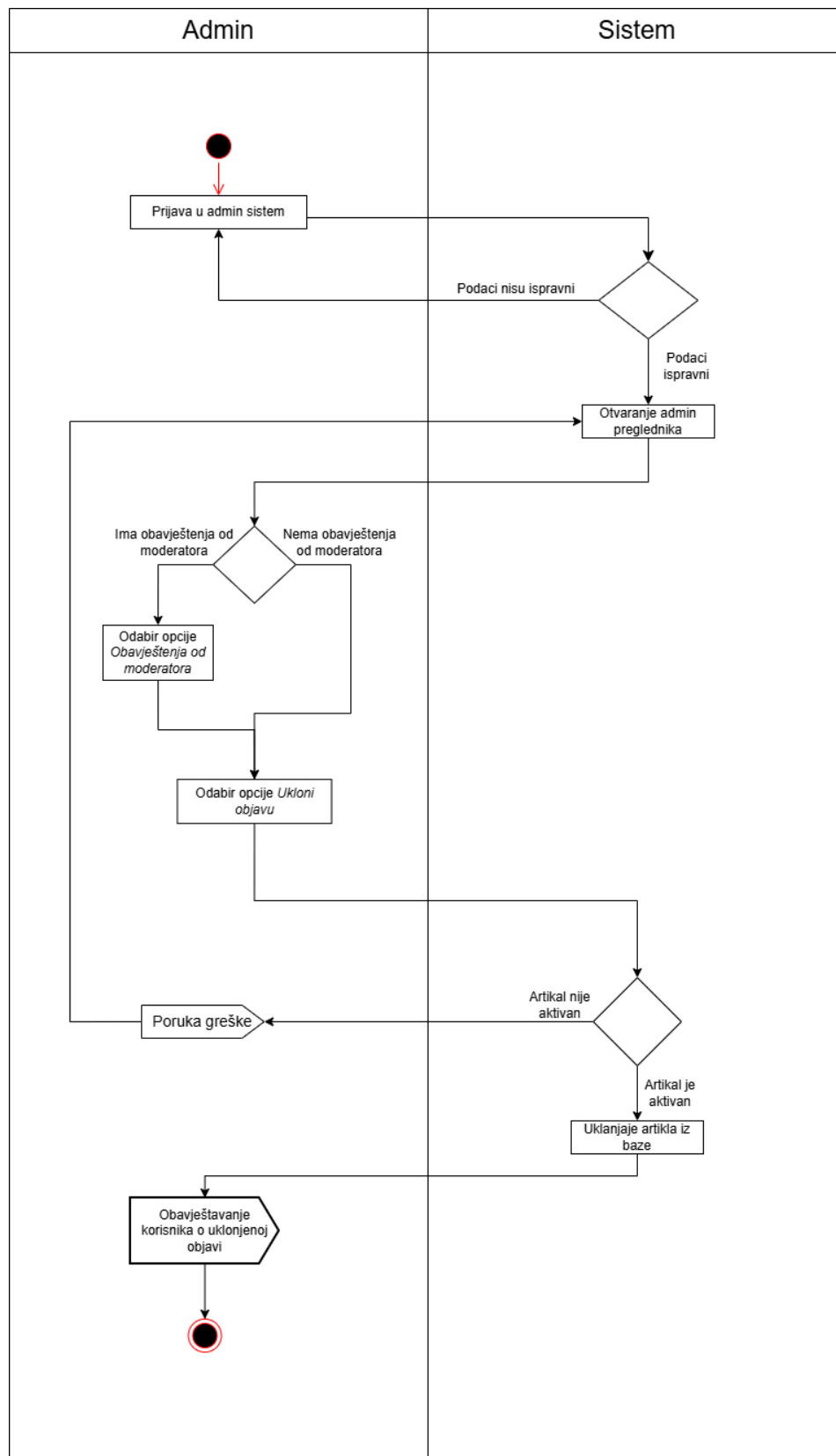
Scenario 5: Pregled i sortiranje oglasa



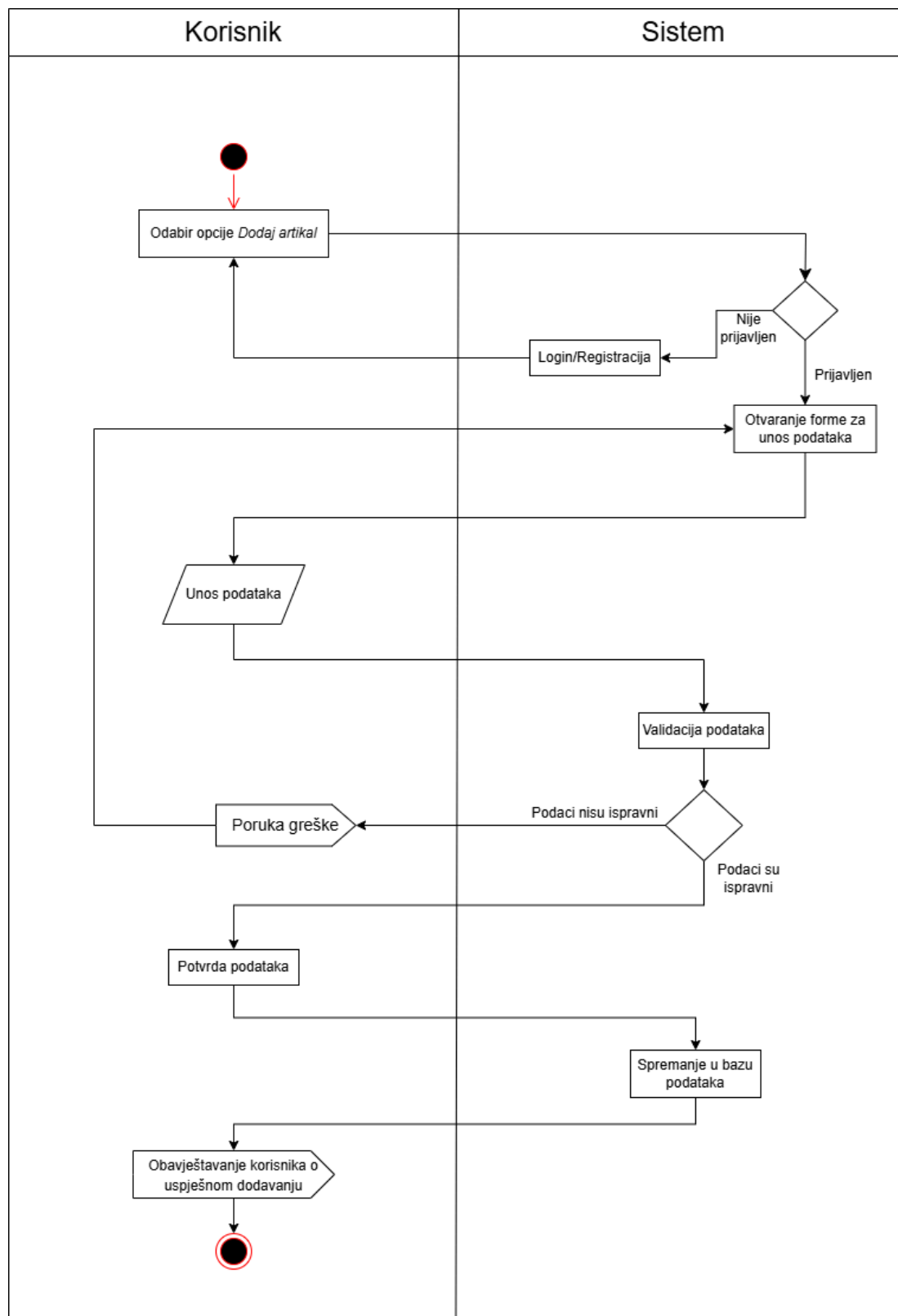
Scenario 6: Moderacija artikala i korisnika



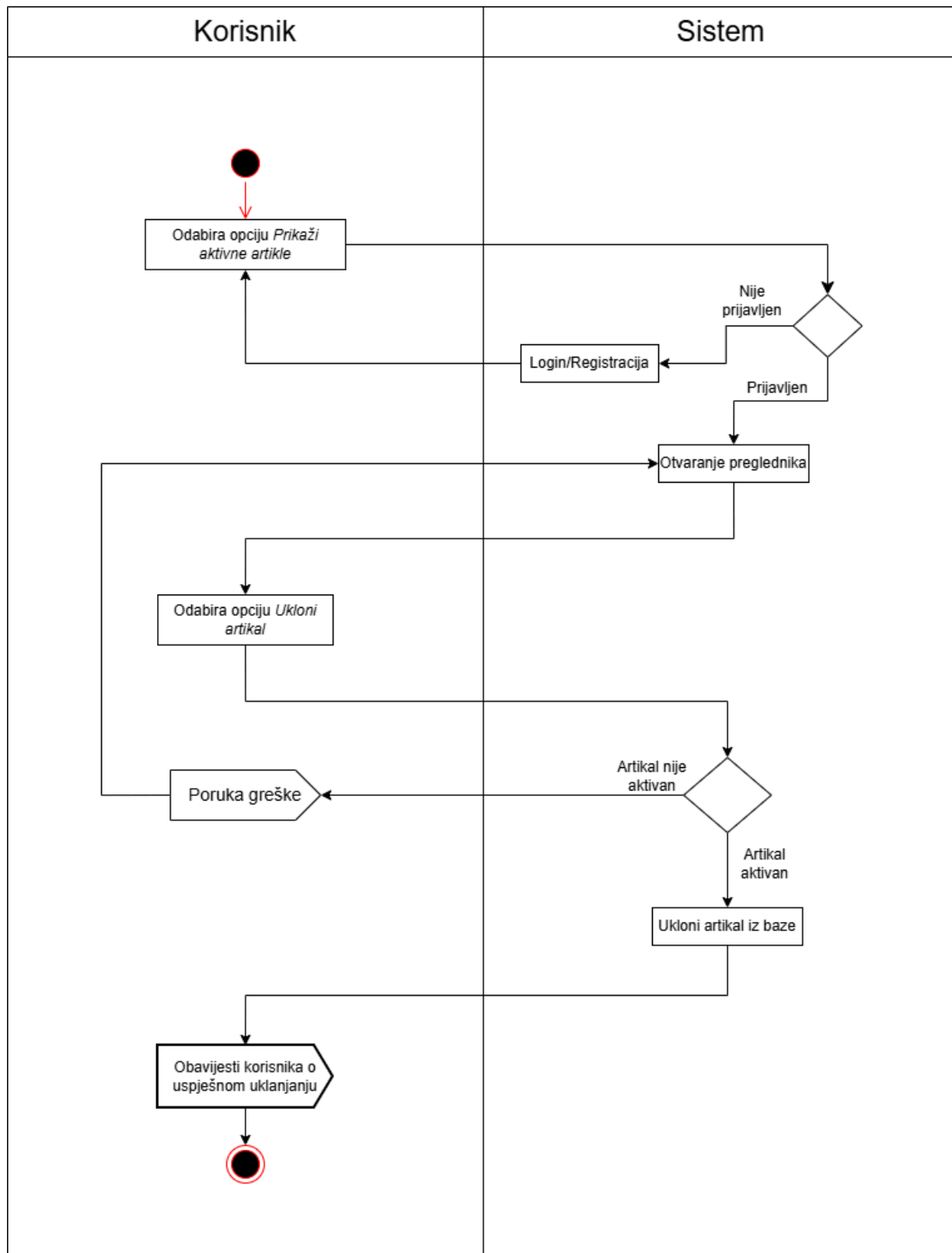
Uklanjanje artikla od strane administratora



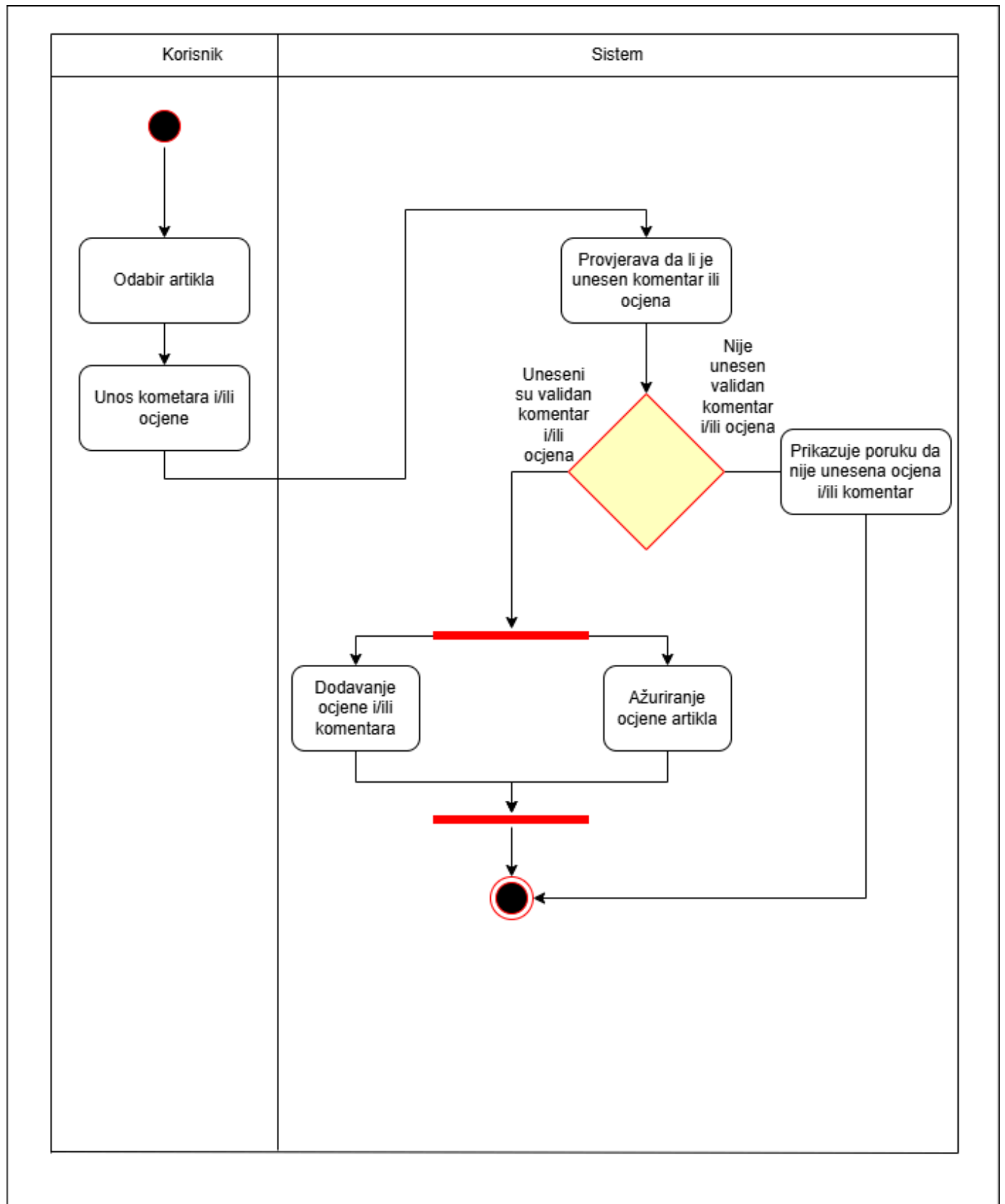
Dodavanje artikla



Uklanjanje artikla



Scenario 10: Ocjenjivanje artikla





Kreirajte svoj nalog

Prijava

Registracija

Ime

Ime

Prezime

Prezime

Email

vase.ime@etf.unsa.ba

Telefon

+387 61 123 456

Lozinka

.....



Potvrdi lozinku

.....



☐ Slažem se sa [uslovima korištenja](#) i [politikom privatnosti](#)

Registruj se

ili

[Nazad na početnu](#)



Dobrodošli nazad!

Prijava

Registracija

Email

vase.ime@etf.unsa.ba

Lozinka

••••••••



☐ Zapamti me

[Zaboravljena lozinka?](#)

Prijavi se

ili

[Nazad na početnu](#)

[← Nazad na oglase](#)

Rezervacija laptopa



MacBook Pro 14" M3
M3 Pro, 18GB RAM, 512GB SSD
Vlasnik: [Noa P.](#) ★ 4.8

📅 Period iznajmljivanja

Datum preuzimanja

mm/dd/yyyy



Datum vraćanja

mm/dd/yyyy



🕒 Vrijeme preuzimanja

09:00 - 10:00



💰 Način plaćanja

☒ Gotovina pri preuzimanju☐ Bankovna transakcija

Dodatne napomene (opciono)

Napišite bilo kakve dodatne zahtjeve ili pitanja...

Rezime rezervacije

Cijena po danu	30 KM
Broj dana	7
Depozit	100 KM

Ukupno 310 KM[Potvrdi rezervaciju](#)

Depozit će biti vraćen nakon povratka laptopa

Pitanja o laptopu?

[Kontaktiraj vlasnika](#)



Noa P.

★★★★★ 4.8 (24 ocjena)

✉ noa.p@etf.unsa.ba

☎ +387 61 234 567

📍 Sarajevo, BiH

📅 Član od Januar 2025

12

Aktivnih oglasa

18

Završenih poslova

Uredi profil

🔍 Pogledaj ocjene

Moji oglasi

[Pogledaj sve](#)

Prodaja



Lenovo ThinkPad X1

i5 10th gen - 16GB - 512GB SSD

350 KM



Rijad D.

★ 5.0

Kontaktiraj prodavača

Iznajmljivanje



Dell XPS 13

i7 - 16GB RAM - 512GB SSD

15 KM / dan



Harun Dž.

★ 4.2

Iznajmi / Kalendar

Prodaja



HP EliteBook 840

Ryzen 5 - 8GB - 256GB SSD

Iznajmljivanje



MacBook Air M1

Apple M1 - 8GB - 256GB

[← Nazad](#)

Ocijeni korisnika



MacBook Pro 14" M3

Iznajmljivanje

Datum: 15.04.2026



Noa P.

★★★★★ 4.8 (trenutna ocjena)

Tvoja ocjena



Brza ocjena (opciono)

☐ Pouzdan☐ Brza komunikacija☐ Tačan opis☐ Laptop u odličnom stanju☐ Preporučujem☐ Fer cijena

Napiši recenziju (opciono)

Podijeli svoje iskustvo sa ovim korisnikom...

Prethodne ocjene

Marko S.

10.04.2026



Odličan laptop, tačno kao što je opisano. Prodavac veoma kooperativan i pouzdan!

Ana K.

05.04.2026



Sve pohvale! Brza komunikacija i laptop u savršenom stanju.

Emir D.

28.03.2026



Dobar laptop, jedino mi je dostava trajala malo duže nego što je dogovoreno.

[Pogledaj sve ocjene →](#)

Svi oglasi

Prodaja

Iznajmljivanje

Moji oglasi

Sortiraj: Po ocjeni prodavača ▾

Cijena: Sve cijene ▾

Sve

Novo

Polovno

Dostupni laptopi

8 oglasa

Prodaja



Lenovo ThinkPad X1

i5 10th gen - 16GB - 512GB SSD

350 KM



Rijad D.

★ 5.0

Kontaktiraj prodavača

Iznajmljivanje



Dell XPS 13

i7 - 16GB RAM - 512GB SSD

15 KM / dan



Harun Dž.

★ 4.2

Iznajmi / Kalendar

Prodaja



HP EliteBook 840

Ryzen 5 - 8GB - 256GB SSD

220 KM

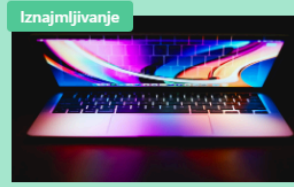


Ali H.

★ 4.7

Kontaktiraj prodavača

Iznajmljivanje



MacBook Air M1

Apple M1 - 8GB - 256GB

20 KM / dan



Noa P.

★ 5.0

Iznajmi / Kalendar

Prodaja



Iznajmljivanje



Prodaja



Iznajmljivanje



Svi oglasi

Prodaja

Iznajmljivanje

Sortiraj: Po ocjeni prodavača ▾

Cijena: Sve cijene ▾

Sve

Novo

Polovno

Dostupni laptopi

8 oglasa

Prodaja



Lenovo ThinkPad X1

i5 10th gen - 16GB - 512GB SSD

350 KM



Rijad D.

★ 5.0

Kontaktiraj prodavača

Iznajmljivanje



Dell XPS 13

i7 - 16GB RAM - 512GB SSD

15 KM / dan



Harun Dž.

★ 4.2

Iznajmi / Kalendar

Prodaja



HP EliteBook 840

Ryzen 5 - 8GB - 256GB SSD

220 KM

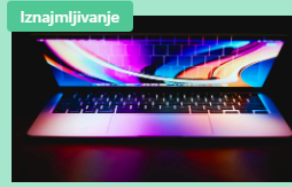


Ali H.

★ 4.7

Kontaktiraj prodavača

Iznajmljivanje



MacBook Air M1

Apple M1 - 8GB - 256GB

20 KM / dan



Noa P.

★ 5.0

Iznajmi / Kalendar

Prodaja



Iznajmljivanje



Prodaja



Iznajmljivanje



[← Nazad na profil](#)

Uredi profil



Klikni na ikonicu da promijeniš sliku

Lični podaci

Ime

Noa

Prezime

P.

Kontakt informacije

Email

noa.p@etf.unsa.ba

Telefon

+387 61 234 567

Adresa

Ulica i broj

Zmaja od Bosne bb

[← Nazad](#)

Uredi oglas

Tip oglasa

☒ Prodaja

☐ Iznajmljivanje

Slike



Maksimalno 6 slika

Osnovne informacije

Naslov oglasa

MacBook Pro 14" M3

Brend

Apple

Model

MacBook Pro 14"

Specifikacije

Procesor

RAM (GB)

Svi oglasi

Prodaja

Iznajmljivanje

Admin panel — upravljanje oglasima

Možete ukloniti oglase koji krše uslove korištenja

3 na čekanju

Sortiraj:

Po ocjeni prodavača

Cijena:

Sve cijene

Sve

Novo

Polovno

Dostupni laptopi

8 oglasa

Prodaja

Ukloni



Lenovo ThinkPad X1

i5 10th gen - 16GB - 512GB SSD

350 KM

Rijad D.

★ 5.0

Kontaktiraj prodavača

Iznajmljivanje

Ukloni



Dell XPS 13

i7 - 16GB RAM - 512GB SSD

15 KM / dan

Harun Dž.

★ 4.2

Iznajmi / Kalendar

Prodaja

Ukloni



HP EliteBook 840

Ryzen 5 - 8GB - 256GB SSD

220 KM

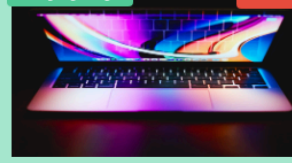
Ali H.

★ 4.7

Kontaktiraj prodavača

Iznajmljivanje

Ukloni



MacBook Air M1

Apple M1 - 8GB - 256GB

20 KM / dan

Noa P.

★ 5.0

Iznajmi / Kalendar

Objavi novi artikal

Slika proizvoda



Klikni za upload slike

PNG, JPG do 5MB

Tip oglasa

Prodaja

Iznajmljivanje

Detalji proizvoda

Naziv modela

npr. Lenovo ThinkPad X1

Procesor

i5 10th gen

RAM

16GB

Storage

512GB SSD

Grafička

Intel UHD

Cijena

350

KM

Stanje

Novo

Polovno

Opis

Detaljno opišite laptop...

Kontaktiraj prodavača

Objavi oglas

[← Nazad na sve oglase](#)

Prodaja

Polovno

Lenovo ThinkPad X1 Carbon

1850 KM

Opis

Izvršno očuvan Lenovo ThinkPad X1 Carbon. Laptop je korišten za kancelarijski rad, bez oštećenja. Baterija drži 6-7 sati normalnog rada. Dolazi sa originalnim punjačem i torbom. Idealan za poslovne korisnike koji traže pouzdanost i mobilnost.

Procesor

Intel Core i7 11th Gen

RAM

16GB DDR4

Storage

512GB NVMe SSD

Grafička kartica

Intel Iris Xe

Cijena

1850 KM

Kupujem

Kontaktiraj prodavača

Informacije o prodavcu



Prodavac

Marko Petrović



Lokacija

Sarajevo, Centar



Telefon

+387 65 123 456



Email

marko.petrovic@email.com

Prednosti

- ✓ Brza dostava
- ✓ Mogućnost pregleda
- ✓ Sigurna kupovina
- ✓ Proverjeni prodavac

Detaljne specifikacije

Procesor

Intel Core i7-1165G7 (2.8GHz - 4.7GHz)

RAM

16GB DDR4 3200MHz

Analiza i dizajn sistema

U nastavku su definirane sve potencijalne klase koje se koriste u sistemu. Za određivanje klasa korišteni su scenariji projekta (slučajevi upotrebe) koji opisuju funkcionalnosti sistema.

Sistem obuhvata 11 funkcionalnih zahtjeva raspoređenih u 10 klasa.

Pregled funkcionalnih zahtjeva

ID	Naziv funkcionalnog zahtjeva
FZ01	Registracija korisnika
FZ02	Prijava korisnika (Login)
FZ03	Prodaja artikla (Objavljivanje oglasa)
FZ04	Iznajmljivanje artikla
FZ05	Kontaktiranje prodavača
FZ06	Pregled i sortiranje oglasa
FZ07	Moderacija artikala i korisnika
FZ08	Administracija (uklanjanje objava)
FZ09	Dodavanje artikla
FZ10	Uklanjanje artikla
FZ11	Ocjenjivanje artikla

Definicija klasa u sistemu

Klasa: Korisnik

Funkcionalni zahtjevi u kojima klasa učestvuje:

- (FZ01: Registracija korisnika)
- (FZ02: Prijava korisnika (Login))
- (FZ03: Prodaja artikla (Objavljivanje oglasa))
- (FZ04: Iznajmljivanje artikla)
- (FZ05: Kontaktiranje prodavača)
- (FZ06: Pregled i sortiranje oglasa)

(FZ09: Dodavanje artikla)
(FZ10: Uklanjanje artikla)
(FZ11: Ocjenjivanje artikla)

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
korisnikId	int	—
ime	String	—
prezime	String	—
email	String	—
sifra	String	—
uloga	UlogaEnum	✓ Atribut je enumeracija
datumRegistracije	Date	—
aktivan	boolean	—

Klasa: Artikel

Funkcionalni zahtjevi u kojima klasa učestvuje:

(FZ03: Prodaja artikla (Objavljivanje oglasa))
(FZ04: Iznajmljivanje artikla)
(FZ06: Pregled i sortiranje oglasa)
(FZ07: Moderacija artikala i korisnika)
(FZ08: Administracija (uklanjanje objava))
(FZ09: Dodavanje artikla)
(FZ10: Uklanjanje artikla)
(FZ11: Ocjenjivanje artikla)

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
artikelId	int	—
naziv	String	—
opis	String	—

cijena	double	—
stanje	StanjeEnum	✓ Atribut je enumeracija
tip	TipArtiklaEnum	✓ Atribut je enumeracija
datumObjave	Date	—
aktivan	boolean	—
prosjecnaOcjena	double	—
kategorijald	int	—

Klasa: SlikaArtikla

Funkcionalni zahtjevi u kojima klasa učestvuje:

(FZ03: Prodaja artikla (Objavljivanje oglasa))

(FZ09: Dodavanje artikla)

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
slikald	int	—
putanjaDatoteke	String	—
artikalId	int	—

Klasa: Rezervacija

Funkcionalni zahtjevi u kojima klasa učestvuje:

(FZ04: Iznajmljivanje artikla)

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
rezervacijald	int	—

datumOd	Date	—
datumDo	Date	—
status	StatusRezervacijeEnum	✓ Atribut je enumeracija
datumZahtjeva	Date	—
korisnikId	int	—

Klasa: Ocjena

Funkcionalni zahtjevi u kojima klasa učestvuje:

(FZ11: Ocjenjivanje artikla)

Naziv atributa	Tip varijable	Dodatne napomene
ocjenaId	int	—
vrijednost	int	—
datumOcjenjivanja	Date	—
artikalId	int	—
korisnikId	int	—

Klasa: Primjedba

Funkcionalni zahtjevi u kojima klasa učestvuje:

(FZ07: Moderacija artikala i korisnika)

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
primjedbaId	int	—
sadržaj	String	—

datumPrimjedbe	Date	—
validna	boolean	—
artikalId	int	—
korisnikId	int	—

Klasa: ZahtjevZaUklanjanje

Funkcionalni zahtjevi u kojima klasa učestvuje:

- (FZ07: Moderacija artikala i korisnika)
- (FZ08: Administracija (uklanjanje objava))

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
zahtjevId	int	—
razlog	String	—
datumZahtjeva	Date	—
status	StatusZahtjevaEnum	✓ Atribut je enumeracija
artikalId	int	—
korisnikId	int	—
adminModeratorId	int	—

Klasa: AdminModerator

Funkcionalni zahtjevi u kojima klasa učestvuje:

- (FZ07: Moderacija artikala i korisnika)
- (FZ08: Administracija (uklanjanje objava))
- (FZ10: Uklanjanje artikla)

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
adminModeratorId	int	—
ime	String	—
email	String	—
uloga	UlogaEnum	✓ Atribut je enumeracija

Klasa: Transakcija

Funkcionalni zahtjevi u kojima klasa učestvuje:

- (FZ03: Prodaja artikla (Objavljivanje oglasa))
- (FZ04: Iznajmljivanje artikla)

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
transakcijaID	int	—
artikalID	String	—
kupacID	String	—
prodavacID	String	—
tipTransakcije	tipArtiklaEnum	✓ Atribut je enumeracija
status	StatusEnum	✓ Atribut je enumeracija
datum	Date	—
iznos	double	—

Klasa: Notifikacija

Funkcionalni zahtjevi u kojima klasa učestvuje:

- (FZ01: Registracija korisnika)
- (FZ02: Prijava korisnika (Login))

(FZ03: Prodaja artikla (Objavljivanje oglasa))
(FZ04: Iznajmljivanje artikla)
(FZ09: Dodavanje artikla)
(FZ10: Uklanjanje artikla)
(FZ11: Ocjenjivanje artikla)

Atributi koje klasa posjeduje:

Naziv atributa	Tip varijable	Dodatne napomene
notifikacijaID	int	—
korisnikID	int	—
tip	tipPorukeEnum	✓ Atribut je enumeracija
sadržaj	String	—
datum	Date	—
uloga	UlogaEnum	✓ Atribut je enumeracija

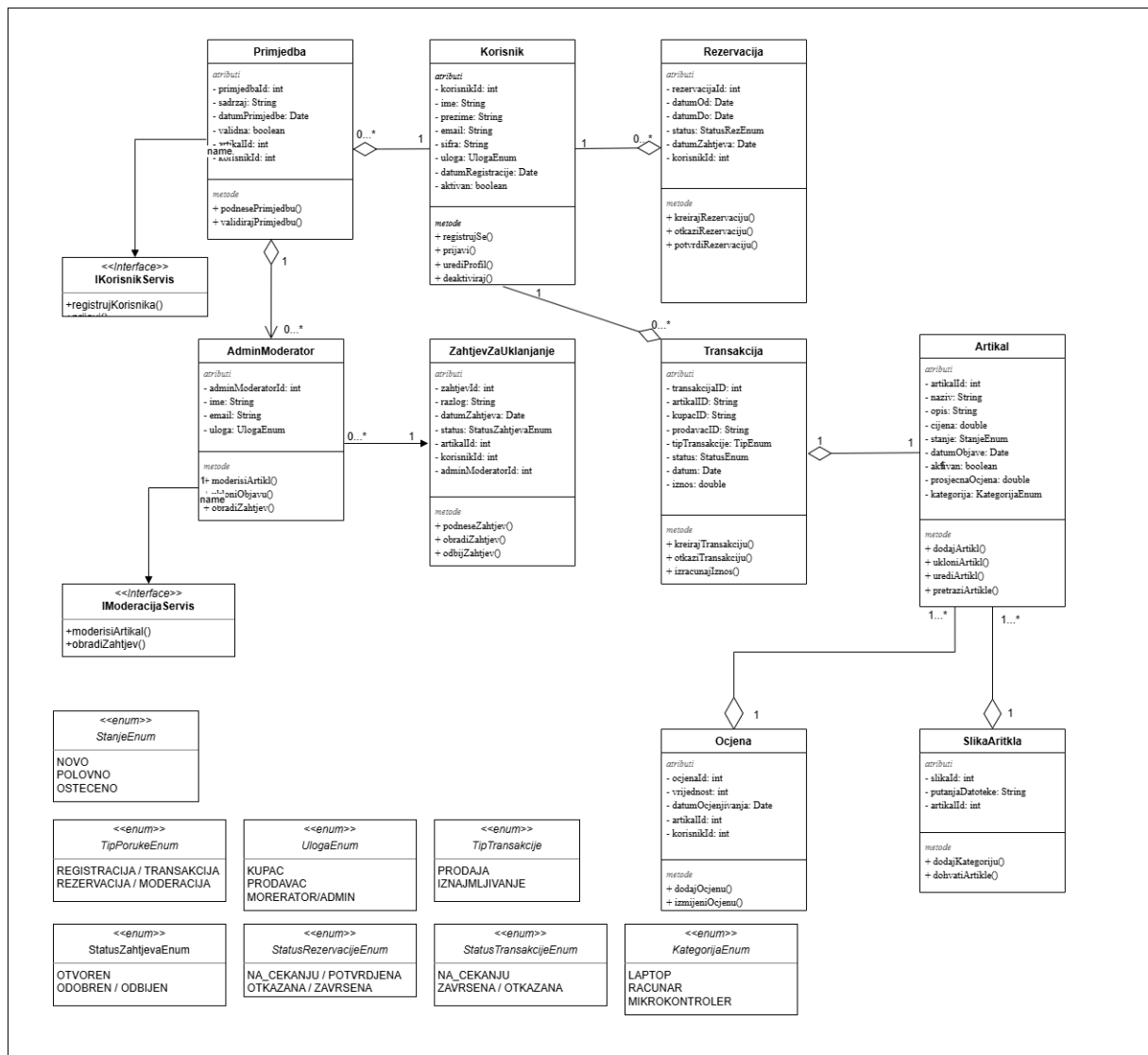
Klasa: Kategorija

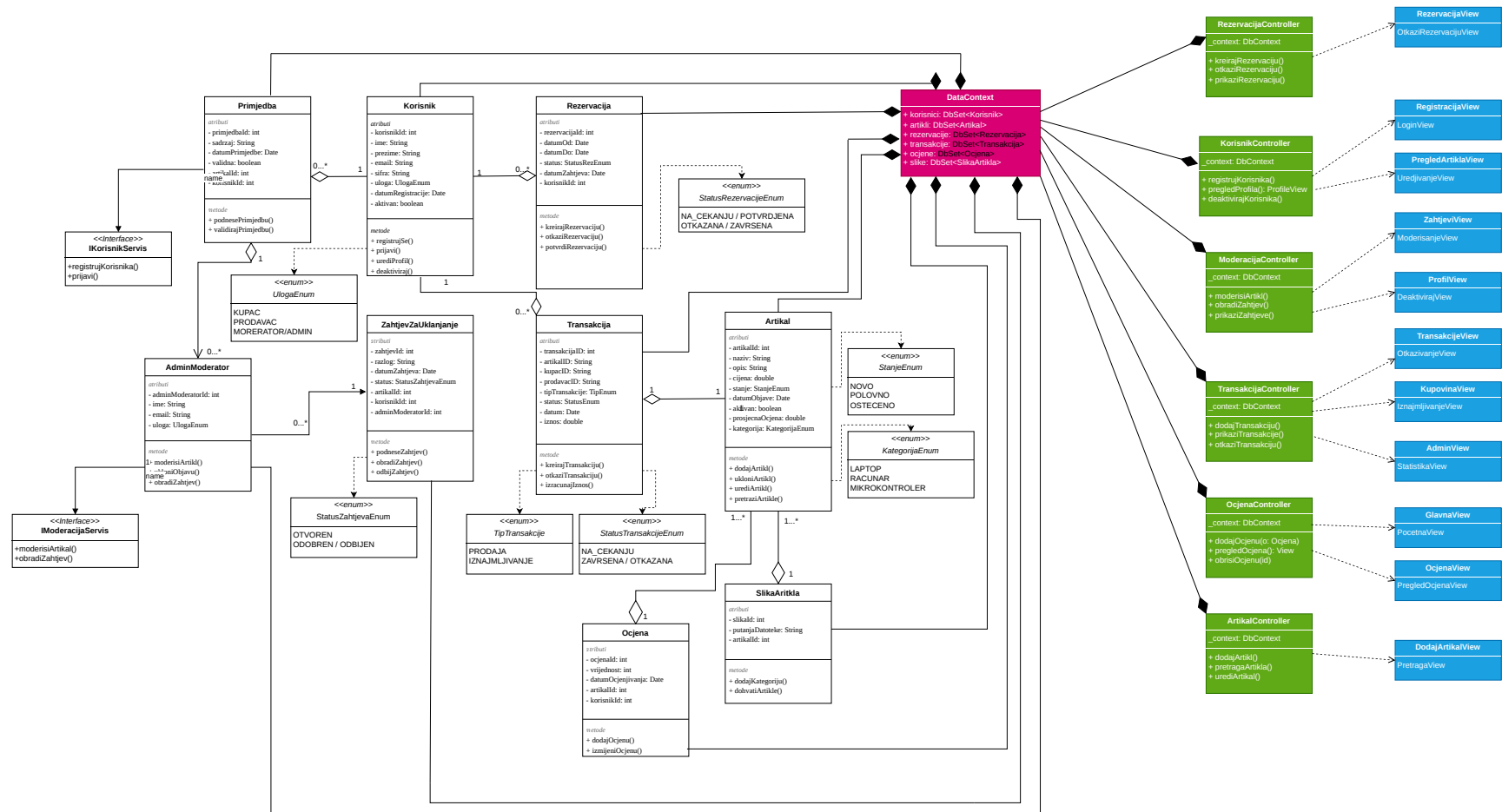
Funkcionalni zahtjevi u kojima klasa učestvuje:

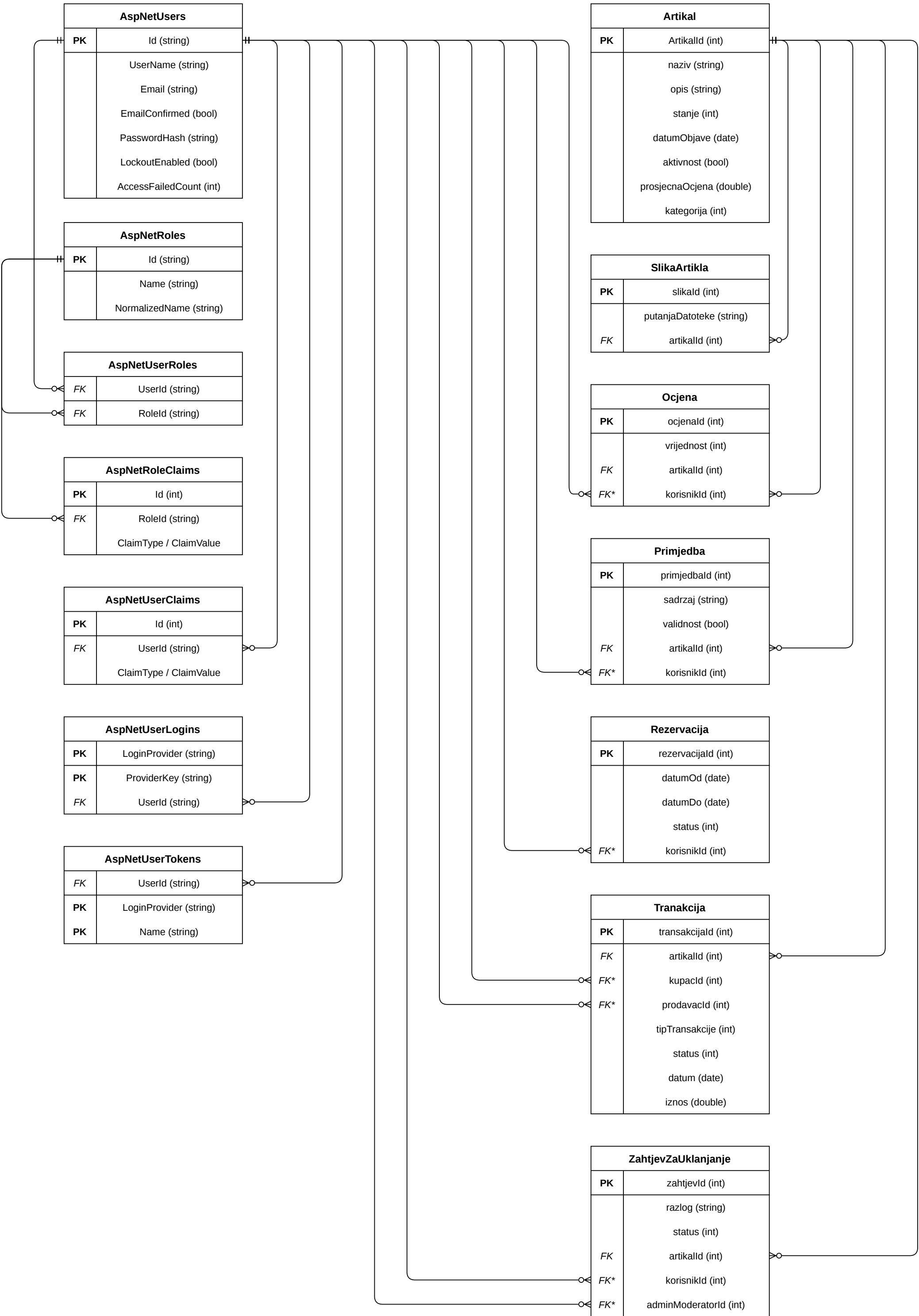
(FZ03: Prodaja artikla (Objavljivanje oglasa))
(FZ04: Iznajmljivanje artikla)
(FZ09: Dodavanje artikla)
(FZ10: Uklanjanje artikla)
(FZ06: Pregled i sortiranje oglasa)

Atributi koje klasa posjeduje:

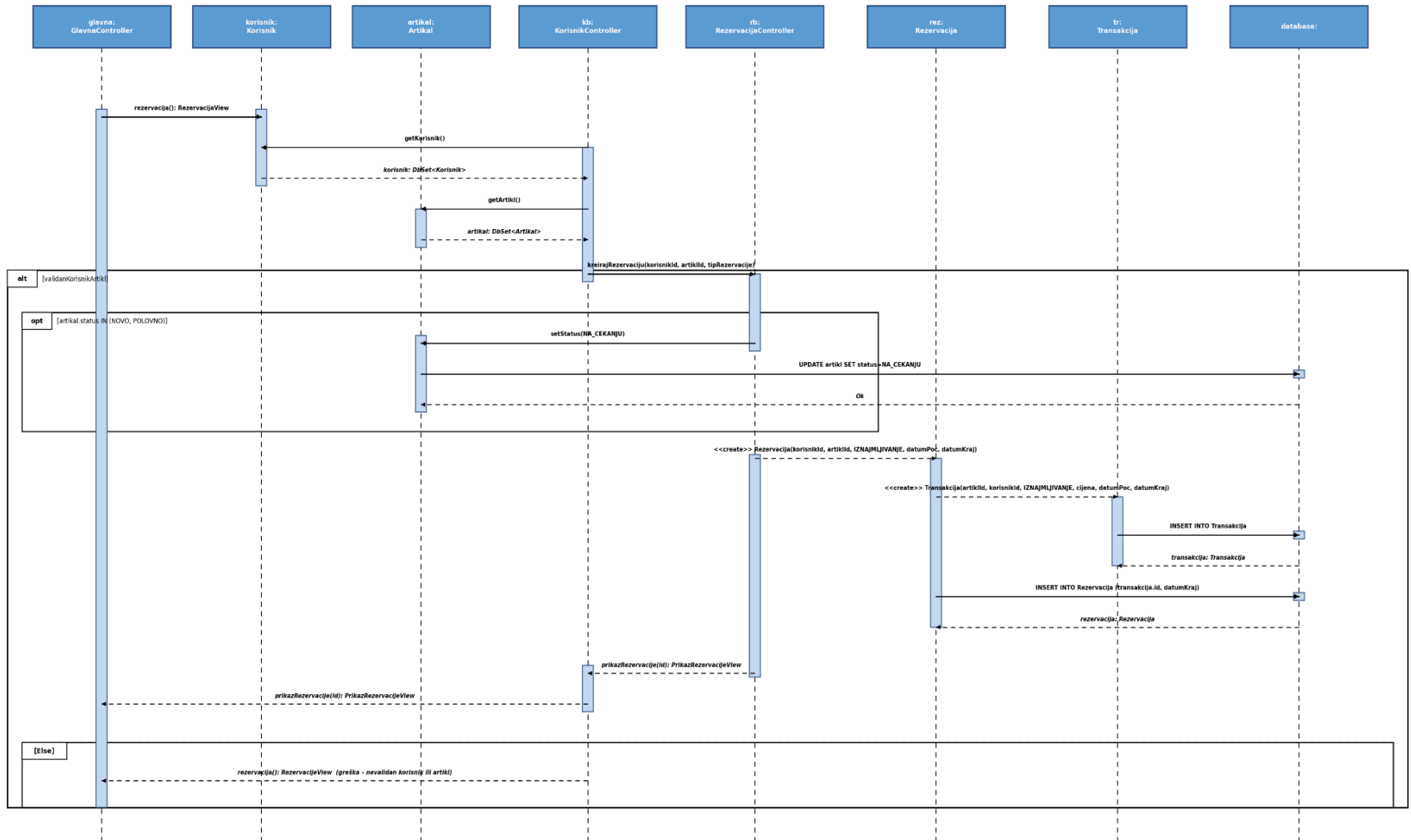
Naziv atributa	Tip varijable	Dodatne napomene
kategorijaID	int	—
naziv	String	—
opis	String	—

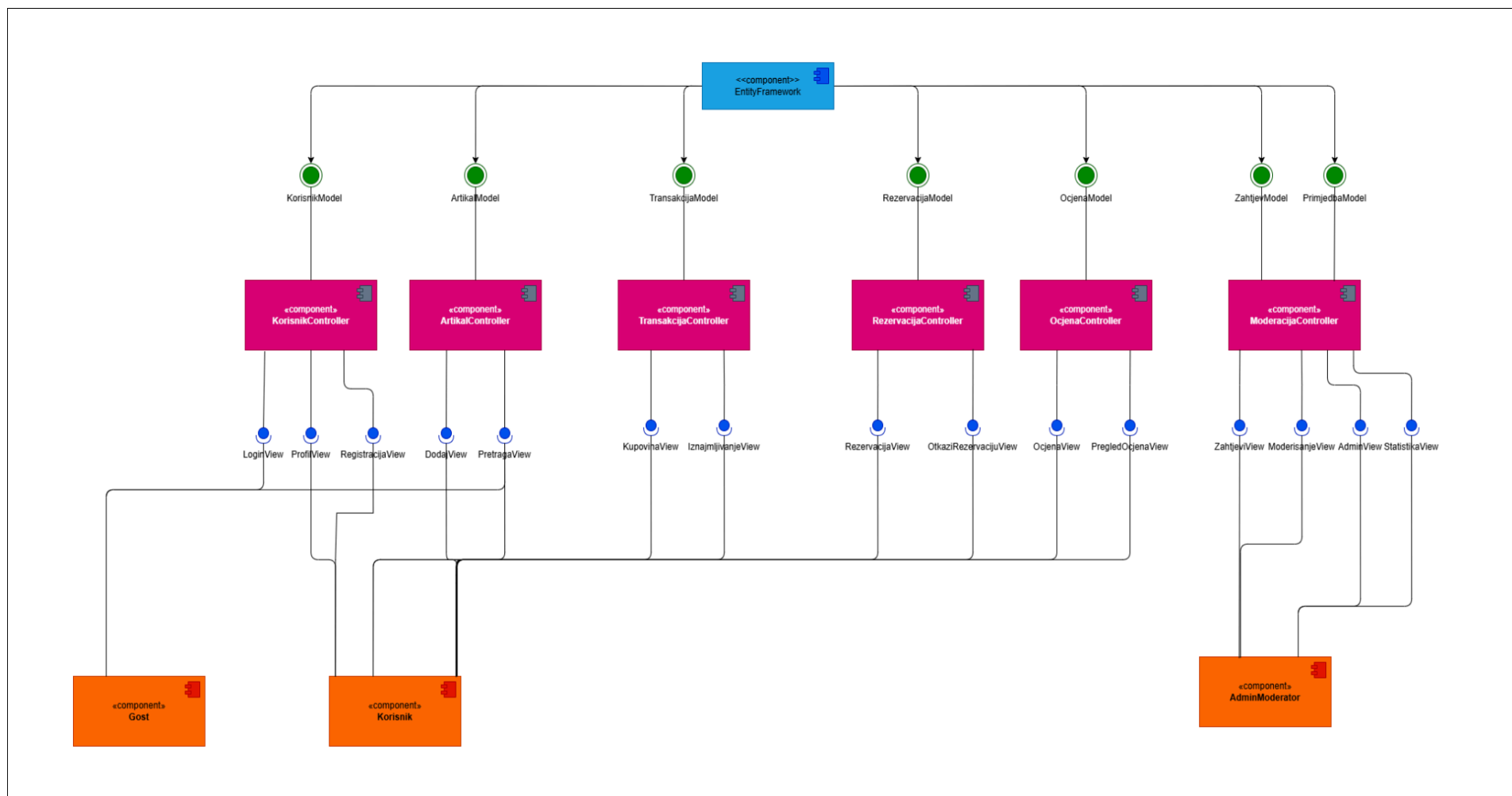


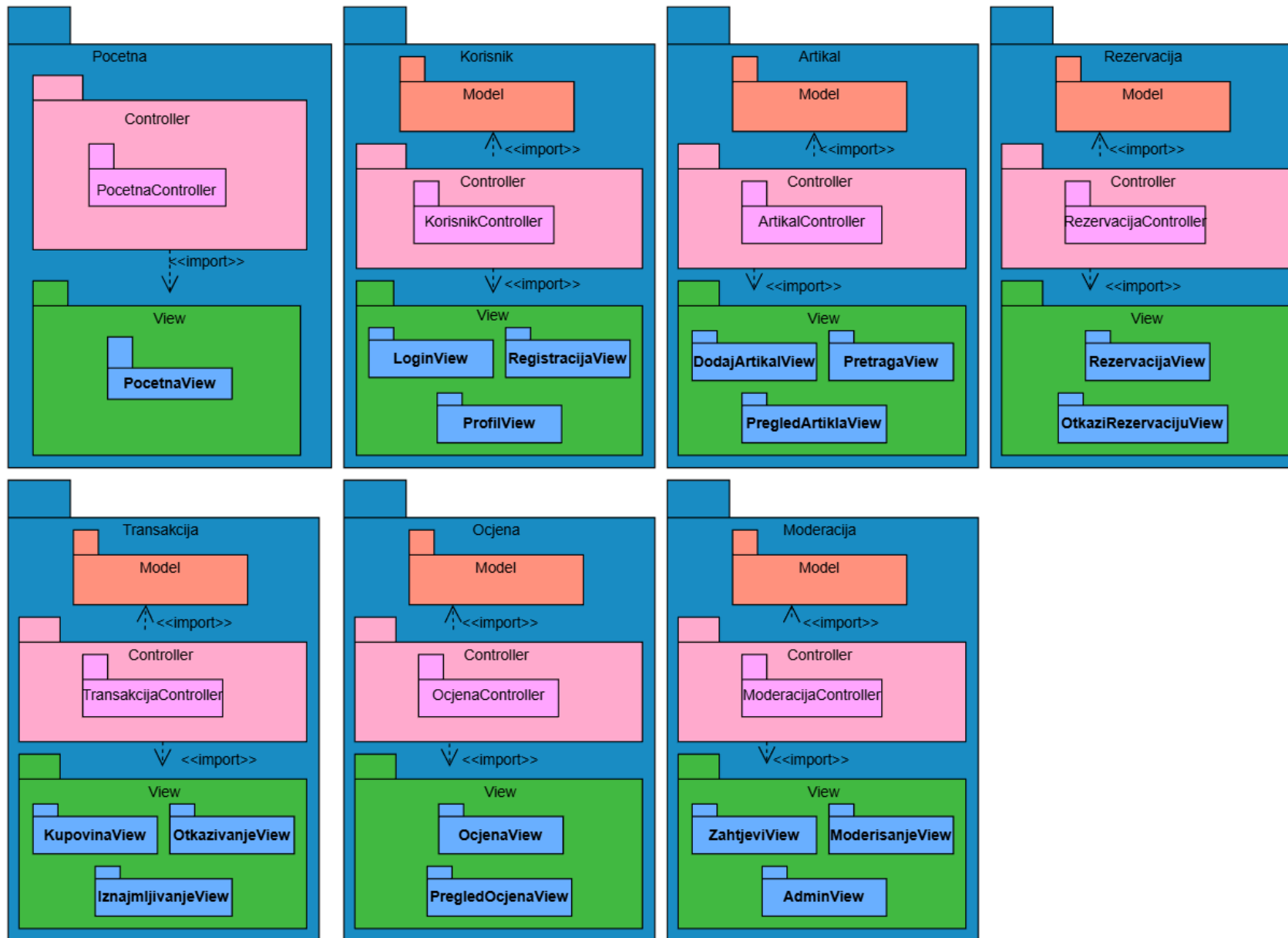


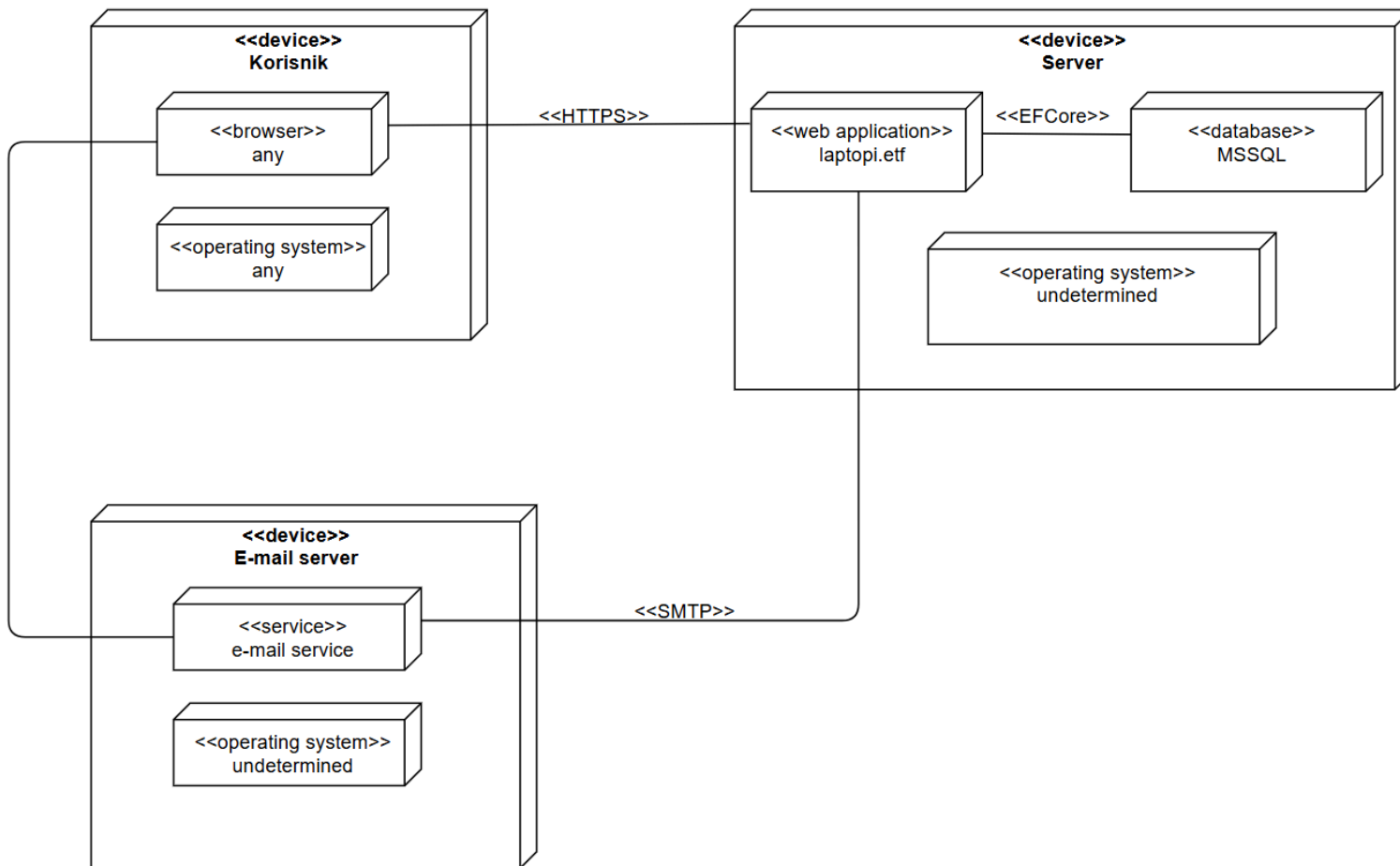


Dijagram Sekvence - Kreiranje Rezervacije (Iznajmljivanje)









1. Single Responsibility Principle (SRP)

Klase: Korisnik, Rezervacija, Transakcija, Artikal, Ocjena, Prijedba, ZahtjevZaUklanjanje

Svaka od ovih klasa ima jasno definisanu jednu odgovornost. Na primjer, Korisnik upravlja podacima i operacijama vezanim za korisnika, dok Transakcija obrađuje isključivo kupoprodajne procese. Time se izbjegava miješanje logike i olakšava održavanje sistema.

2. Open/Closed Principle (OCP)

Klase: Korisnik, AdminModerator, enumeracije (UlogaEnum, StatusTransakcijeEnum, itd.)

Sistem je otvoren za proširenje, a zatvoren za izmjene kroz korištenje enumeracija. Dodavanje novih statusa ili tipova ne zahtijeva izmjenu postojećeg koda, već samo proširenje enumeracija ili novih klasa.

3. Liskov Substitution Principle (LSP)

U ovom slučaju se ne koristi nasljeđivanje.

4. Interface Segregation Principle (ISP)

Interfejsi: IKorisnikServis, IModeracijaServis

Interfejsi su podijeljeni prema odgovornostima - jedan za korisničke operacije, drugi za moderaciju. Klase implementiraju samo one metode koje su im potrebne, čime se izbjegava "prisilna" implementacija nepotrebnih funkcionalnosti.

5. Dependency Inversion Principle (DIP)

Klase: IKorisnikServis, IModeracijaServis, zavisne klase (AdminModerator, itd.)

Visokonivojske komponente zavise od apstrakcija (interfejsa), a ne od konkretnih implementacija. Na primjer, moderacija koristi IModeracijaServis, što omogućava fleksibilnu zamjenu implementacije bez uticaja na ostatak sistema.

Strukturalni paterni

U ovom dokumentu navedeni su strukturalni paterni korišteni u modifikaciji trenutnog dijagrama sistema klasa i razlozi za njihov odabir.

1. Facade patern - KupoprodajaFacade

Problem

U sistemu za kupoprodaju i iznajmljivanje artikala, klijentski kod (npr. kontroler u prezentacijskom sloju) koji želi izvršiti operaciju kupovine mora samostalno koordinirati između više klasa: provjeriti dostupnost `Artikal` objekta, kreirati `Transakcija` objekat, eventualno kreirati `Rezervacija` objekat, te validirati `Korisnik` koji vrši transakciju. Ovo znači da klijent mora poznavati internu strukturu svih tih klasa i njihove međusobne zavisnosti, što rezultira čvrstom spregom (tight coupling) između prezentacijskog sloja i poslovne logike.

Rješenje

Uvedena je klasa `KupoprodajaFacade` koja implementira interfejs `ITransakcijaFacade` i nudi samo dvije jasne, visokonivojske metode prema klijentu: `kupiArtikl()` i `iznajmiArtikl()`. Unutar sebe, Facade interno koordinira sve potrebne korake - provjerava korisnika, provjerava dostupnost artikla, kreira transakciju i po potrebi kreira rezervaciju. Klijent ne zna ništa o tim klasama i ne ovisi direktno o njima, već komunicira isključivo kroz Facade interfejs.

Zašto je primjenljiv u ovom sistemu

Sistem već razlikuje dva tipa transakcije kroz `TipTransakcijeEnum` (prodaja i iznajmljivanje), što znači da logika unutar `kupujArtikl()` i `iznajmiArtikl()` prirodno postoji u domeni - Facade je samo organizuje na jednom mjestu. Bez Facade paterna, svaki novi klijent (mobilna aplikacija, web interfejs) morao bi sam reimplementirati tu koordinacijsku logiku.

Konsekvence

Facade smanjuje spregu između slojeva sistema i čini prezentacijski sloj nezavisnim od promjena u poslovnoj logici. Ako se npr. proces kreiranja rezervacije promijeni, mijenja se samo Facade, a ne svaki klijent koji ga koristi. Mana je što Facade može postati tzv. "God object" ako se previše logike akumulira u njemu, pa je važno da samo delegira pozive a ne sadrži kompleksnu logiku sam.

2. Proxy patern - ArtikelServisProxy

Problem

Klasa `Artikel` i operacije nad njom (dodavanje, uređivanje, uklanjanje) nisu dozvoljene svim korisnicima jednako - sistem ima `UlogaEnum` koji razlikuje kupce, prodavače i moderatore, te `aktivan` atribut na `Korisnik` klasi. Međutim, direktan pristup servisnoj logici ne nudi nikakav mehanizam kontrole pristupa. Svaki pozivalac može pozvati bilo koju metodu bez provjere.

Rješenje

Uveden je interfejs `IArtikelServis` koji definira ugovor za sve operacije nad artiklima. Ovaj interfejs implementiraju dvije klase. `ArtikelServisImpl` je tzv. RealSubject - stvarna implementacija koja izvršava operacije nad `Artikel` objektima. `ArtikelServisProxy` je Proxy koji stoji ispred `ArtikelServisImpl`-a: svaki poziv metode prvo prolazi kroz privatnu metodu `provjeriPristup(korisnik)` koja čita `uloga` i `aktivan` attribute iz `Korisnik` objekta. Tek ako je pristup dozvoljen, Proxy delegira poziv pravoj implementaciji. Klijent uvijek razgovara samo sa `IArtikelServis` interfejsom i ne zna da li koristi Proxy ili direktnu implementaciju.

Zašto je primjenljiv u ovom sistemu

Sistem već posjeduje sve potrebne elemente za ovaj patern: `UlogaEnum` definira uloge korisnika, `aktivan: boolean` na `Korisnik` klasi omogućava provjeru aktivnosti, a `ZahtjevZaUklanjanje` i `AdminModerator` klase jasno ukazuju da različiti korisnici imaju različita prava nad artiklima. Proxy centralizuje te provjere na jednom mjestu umjesto da se ponavljaju kroz cijelu aplikaciju.

Konsekvence

Proxy patern omogućava dodavanje kontrole pristupa bez ikakve izmjene u `ArtikelServisImpl` klasi, što je u skladu sa Open/Closed principom - klase su otvorene za proširenje, ali zatvorene za modifikaciju. Klijentski kod se također ne mijenja jer i dalje koristi isti `IArtikelServis` interfejs. Mana je uvođenje dodatnog sloja indirekcije koji blago povećava kompleksnost sistema, ali ta cijena je opravdana sigurnošću i konzistentnošću kontrole pristupa.

3. Adapter patern

Problem

Sistem u budućnosti može imati potrebu za integracijom sa eksternim servisima - npr. eksternim servisom za plaćanje (PayPal, Stripe) ili eksternim servisom za slanje notifikacija (email, SMS). Ti eksterni servisi imaju vlastite interfejske koji nisu kompatibilni sa interfejsima koje naš sistem očekuje. Direktna integracija bi značila mijenjanje postojećih klasa poput `Transakcija` ili `Korisnik` svaki put kada se promijeni eksterni servis, što narušava stabilnost sistema.

Rješenje

Uveo bi se interfejs npr. `IPlacanje` koji naš sistem poznaje i koristi. Za svaki eksterni servis kreirala bi se Adapter klasa, npr. `PayPalAdapter` ili `StripeAdapter`, koja implementira `IPlacanje` interfejs a interno poziva metode eksternog servisa prevodeći pozive iz formata našeg sistema u format koji eksterni servis razumije. Klasa `Transakcija` bi koristila isključivo `IPlacanje` interfejs i ne bi znala ništa o konkretnom eksternom servisu iza njega.

Zašto je primjenjiv u ovom sistemu

Klasa `Transakcija` već sadrži attribute poput `iznos` i `valuta` koji su direktno vezani za proces plaćanja. Kako sistem raste i uvodi više načina plaćanja, Adapter patern omogućava dodavanje novih načina plaćanja bez ikakvog diranja u postojeće klase. Svaki novi servis za plaćanje dobija samo svoju Adapter klasu.

Konsekvence

Adapter patern omogućava da sistem bude neovisan od konkretnih eksternih implementacija, što olakšava zamjenu jednog servisa drugim. Mana je što svaki novi eksterni servis zahtijeva kreiranje nove Adapter klase, što može povećati broj klasa u sistemu, ali je ta cijena opravdana fleksibilnošću i izolacijom promjena.

4. Bridge patern

Problem

U sistemu postoje različite vrste artikala (npr. vozila, nekretnine, elektronika) i različiti načini njihovog prikazivanja ili obrade (prikaz u listi, prikaz u detalju, prikaz na mapi). Ako bi se svaka kombinacija vrste artikla i načina prikaza implementirala kao zasebna klasa, broj klasa bi eksponencijalno rastao - `VoziloListaPrikaz`, `VoziloDetaljPrikaz`, `NekretninaListaPrikaz` itd. Svaka nova vrsta artikla ili novi način prikaza zahtijevao bi kreiranje čitavog novog seta klasa.

Rješenje

Bridge patern razdvaja dvije hijerarhije u zasebne klase koje se razvijaju nezavisno. Apstrakcija bi bila `Artikal` sa podklasama po vrsti (npr. `VoziloArtikl`, `NekretninaArtikl`), dok bi implementacija bila posebna hijerarhija `IPrikazArtikla` sa konkretnim klasama poput `ListaPrikaz` i `DetaljPrikaz`. `Artikal` bi držao referencu na `IPrikazArtikla` interfejs i delegirao mu odgovornost prikaza, umjesto da sam zna kako se prikazuje.

Zašto je primjenjiv u ovom sistemu

Klasa `Artikal` već ima attribute poput `kategorija` i `tipArtikla` koji nagovještavaju da će u sistemu postojati različite vrste artikala. Istovremeno, sistem ima i `SlikaArtikla` klasu koja je već odvojena od same logike artikla, što pokazuje tendenciju sistema ka razdvajanju prikaza od podataka. Bridge patern bi formalizovao taj odnos i proširio ga na cjelokupnu logiku prikaza.

Konsekvence

Bridge patern smanjuje broj potrebnih klasa i omogućava da se nove vrste artikala i novi načini prikaza dodaju potpuno nezavisno jedni od drugih. Mana je povećana inicijalna kompleksnost dizajna jer zahtijeva dobro promišljenu podjelu odgovornosti između dvije hijerarhije, što može biti pretjerano za manji sistem.

5. Composite patern

Problem

U sistemu artikli mogu biti organizirani u kategorije, a kategorije mogu imati podkategorije - npr. "Vozila" → "Automobili" → "Polovni automobili". Klijentski kod koji treba prikazati sve artikle unutar kategorije mora posebno tretirati kategorije koje imaju podkategorije i kategorije koje direktno sadrže artikle, što rezultira složenim `if/else` ili `switch` blokovima i otežava dodavanje novih nivoa hijerarhije.

Rješenje

Uveo bi se zajednički interfejs npr. `IKatalogStavka` koji implementiraju i `Artikal` (list node - nema djece) i `Kategorija` (composite node — može sadržavati druge `IKatalogStavka` objekte, bilo artikle ili podkategorije). Klasa `Kategorija` bi interno čuvala listu `IKatalogStavka` objekata i pri pozivu metode poput `prikazi()` ili `pretrazi()` rekurzivno pozivala istu metodu na svakom svom djetetu. Klijent uvijek radi samo sa `IKatalogStavka` interfejsom i ne razlikuje da li radi sa jednim artiklom ili čitavom kategorijom.

Zašto je primjenjiv u ovom sistemu

Klasa `Artikal` već ima atribut `kategorija` što direktno ukazuje na hijerarhijsku organizaciju artikala u sistemu. Kako sistem raste i broj kategorija se povećava, Composite patern omogućava uniformno upravljanje čitavom hijerarhijom bez posebnog tretmana pojedinih nivoa. Operacije poput pretrage ili filtriranja po kategoriji postaju trivijalne jer se automatski propagiraju kroz hijerarhiju.

Konsekvence

Composite patern čini klijentski kod znatno jednostavnijim jer ne mora razlikovati lisne od kompozitnih čvorova. Dodavanje novih nivoa hijerarhije kategorija ne zahtijeva nikakve izmjene u klijentskom kodu. Mana je što interfejs može postati previše generičan - operacije koje imaju smisla za `Kategorija` ne moraju imati smisla za `Artikal`, pa implementacija interfejsa može biti neprirodan fit za lisne čvorove.

6. Decorator patern

Problem

Artikli u sistemu mogu imati različite dodatne opcije koje prodavač može uključiti — npr. "istaknuti oglas" koji se prikazuje na vrhu liste, "hitna prodaja" koja dodaje posebnu oznaku, ili "verifikovani artikl" koji prikazuje badge povjerenja. Implementacija svih ovih kombinacija kao zasebnih podklasa (`IstaknutiArtikl`, `HitnaProduktArtikl`, `IstaknutiHitniArtikl` itd.) bila bi neodrživa jer broj kombinacija eksponencijalno raste sa svakom novom opcijom.

Rješenje

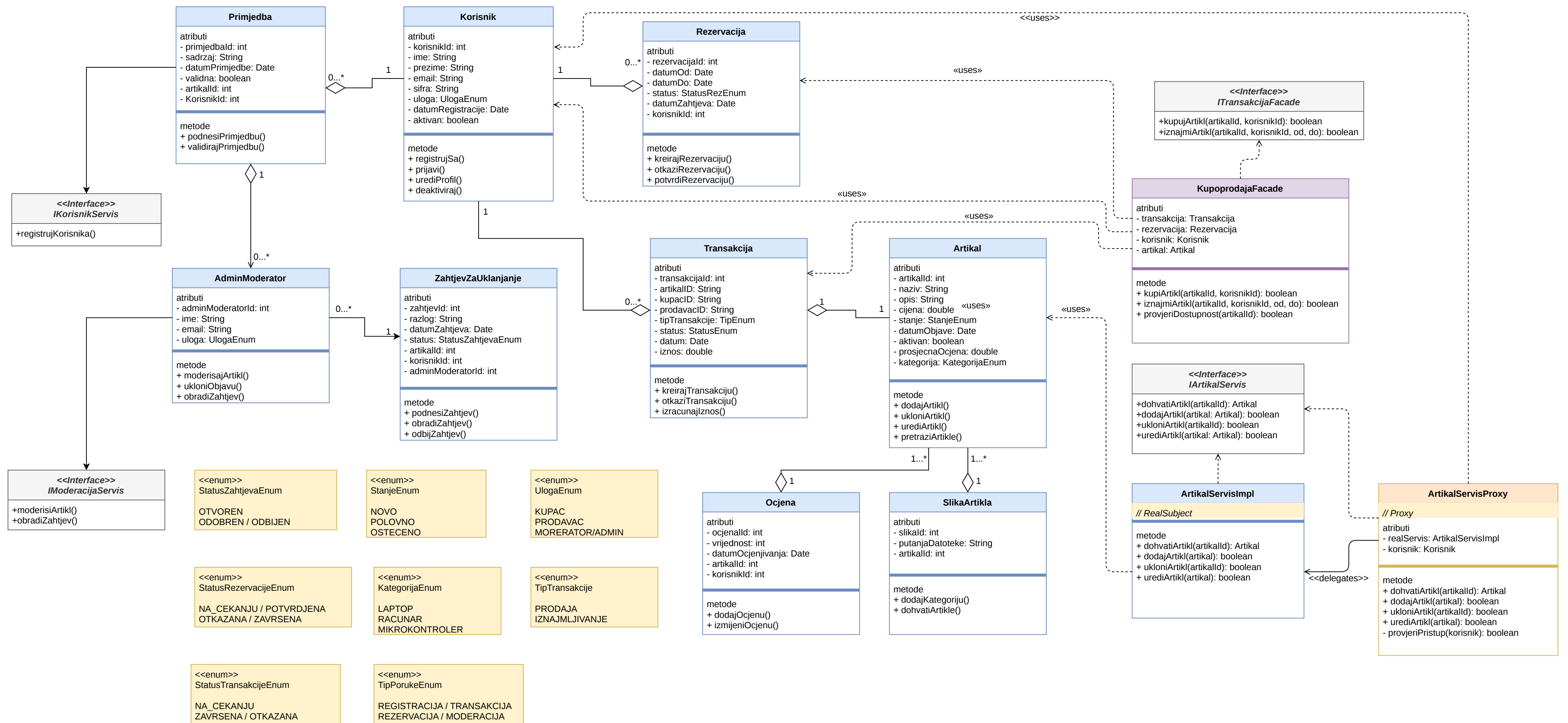
Uveo bi se `ArtikalDecorator` kao apstraktna klasa koja implementira isti interfejs kao i `Artikal` i interno drži referencu na `Artikal` objekat koji dekorira. Konkretni dekoratori poput `IstaknutiArtikalDecorator` ili `VerifikovaniArtikalDecorator` nasljeđuju `ArtikalDecorator`, dodaju svoju specifičnu funkcionalnost (npr. dodavanje oznake ili promjena pozicije u listi) i delegiraju ostale pozive originalnom `Artikal` objektu. Dekoratori se mogu slagati jedan na drugi u bilo kojoj kombinaciji.

Zašto je primjenjiv u ovom sistemu

Sistem već ima `Ocjena` i `Primjedba` klase koje dinamički obogaćuju artikal dodatnim informacijama, a `AdminModerator` i `ZahtjevZaUklanjanje` ukazuju da artikli prolaze kroz različite statusne promjene tokom svog životnog vijeka. Decorator patern bi formalizovao mehanizam dinamičkog dodavanja novih ponašanja artiklima bez nasljeđivanja, što je posebno korisno jer se kombinacije opcija mogu mijenjati u toku izvršavanja programa prema izboru prodavača.

Konsekvence

Decorator patern eliminiše eksploziju podklasa i omogućava dinamičko dodavanje i uklanjanje funkcionalnosti u toku izvršavanja, za razliku od statičnog nasljeđivanja. Mana je što sistem dekoratora može biti teže za debugiranje jer se pozivi propagiraju kroz više slojeva dekoratora, a redosljed kojim se dekoratori slažu može utjecati na krajnji rezultat.



Kreacijski paterni

U sklopu razvoja sistema za kupovinu i iznajmljivanje artikala, identifikovana je potreba za uvođenjem kreacijskih dizajn paterna. Kreacijski paterni se bave procesom kreiranja objekata i pomažu u upravljanju složenošću instanciranja klasa. Odabrani su paterni koji se prirodno uklapaju u postojeću arhitekturu sistema i donose konkretne prednosti u pogledu fleksibilnosti, proširivosti i održavanja koda. Od pet standardnih kreacijskih paterna, za implementaciju su odabrani Prototype i Factory Method, dok preostala tri paterna Singleton, Abstract Factory i Builder nisu implementirana, ali je opisano gdje bi se mogli primijeniti u okviru sistema.

1. Prototype pattern

Osnovna funkcija Prototype paterna je da olakša kreiranje objekata koristeći postojeću instancu, koja se ponaša kao prototip. Novokreiranom objektu možemo promijeniti određene osobine po potrebi.

U našem sistemu za kupovinu i iznajmljivanje artikala, ovaj patern je primjenjen nad klasom **Artikal**. Prodavači često oglašavaju slične artikle koji dijele većinu atributa (naziv, kategorija, stanje, cijena). Umjesto ručnog ponovnog unosa svih podataka, prodavač može klonirati postojeći artikal kao predložak i izmijeniti samo specifične detalje za novi oglas. Na taj način se ubrzava proces kreiranja novih oglasa te se smanjuje mogućnost greške pri unosu.

Klasa **Artikal** implementira interfejs **IPrototype** koji sadrži metodu **clone()**. Ova metoda kreira i vraća kopiju postojećeg artikla. Kontroler **ArtikalController** dobija novu metodu **klonirajArtikla()** koja poziva **clone()** nad odabranim artiklom i vraća novu instancu spremnu za uređivanje

2. Factory Method pattern

Factory Method patern omogućava kreiranje objekata na način da podklase odluče koju klasu instancirati. Ovo se radi preko interfejsa sa jednom metodom koju različite podklase mogu implementirati drugačije.

U našoj aplikaciji, ovaj patern je primjenjen nad klasom **Transakcija**. Sistem podržava dva tipa transakcija definisana kroz enum **TipTransakcije**: **PRODAJA** i **IZNAJMLJIVANJE**. Svaki tip ima svoju logiku i specifične attribute - transakcija iznajmljivanja sadrži dodatne attribute **datumOd** i **datumDo**, dok transakcija prodaje ne zahtijeva te podatke.

Uveli smo interfejs **ITransakcija** sa metodom **kreirajTransakciju()** te klasu **TransakcijaCreator** koja sadrži factory metodu. Podklase **ProdajaTransakcija** i **IznajmljivanjeTransakcija** implementiraju ovaj interfejs svaka na svoj način. **TransakcijaController** koristi **TransakcijaCreator** za kreiranje odgovarajuće transakcije na osnovu korisnikovog odabira, čime se postiže odvajanje logike kreiranja od ostatka sistema

3. Singleton pattern

Uloga Singleton paterna je da osigura da se klasa može instancirati samo jednom i da osigura globalni pristup kreiranoj instanci klase.

U našoj aplikaciji **Singleton** obrazac bi se mogao iskoristiti za upravljanje sistemom popusta po kategorijama proizvoda. Servis za popuste implementiran je kao Singleton kako bi se osigurala jedna zajednička instanca kroz cijelu aplikaciju i konzistentno upravljanje pravilima popusta. Popusti se mjenjaju periodično, te se ne računaju dinamički za svakog korisnika. Cijene proizvoda se uvijek dohvaćaju iz baze podataka, dok Singleton servis za popuste primjenjuje odgovarajući procenat popusta na osnovu kategorije proizvoda prije prikaza krajnje cijene korisniku.

4. Abstract Factory pattern

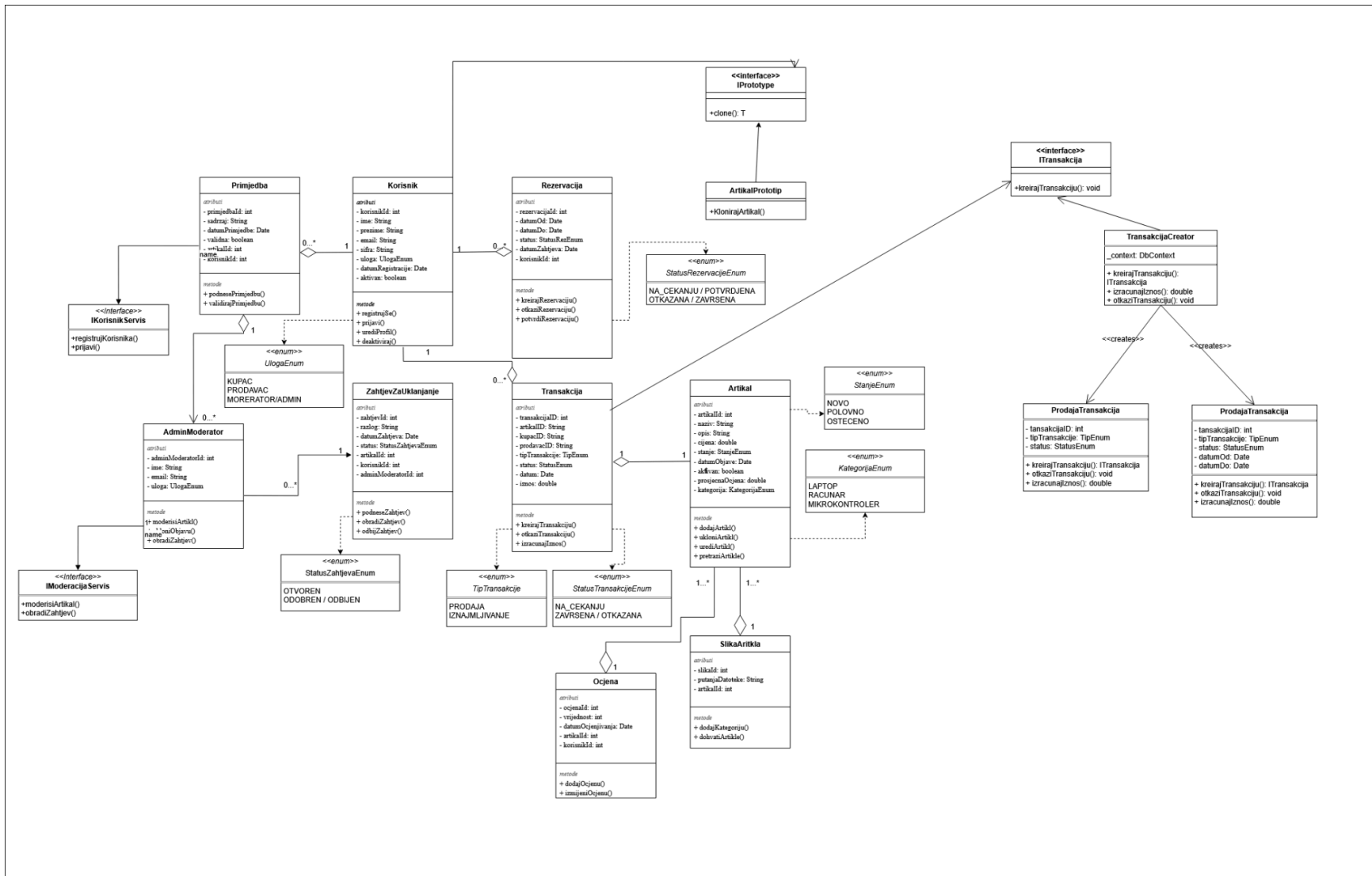
Abstract Factory patern omogućava kreiranje više familija objekata sa mogućnošću dodavanja novih u budućnosti. Abstract Factory odvaja definiciju (klase) produkata od klijenta, zbog čega se familije produkata mogu jednostavno izmijenjivati ili ažurirati bez narušavanja strukture klijenta.

U kontekstu našeg sistema, Abstract Factory bi se mogao primijeniti za upravljanje različitim tipovima korisnika. Sistem trenutno poznaje dvije uloge - **Korisnik** i **AdminModerator** - koje imaju različite skupove dozvola i pristupa funkcionalnostima. Uvođenjem Abstract Factory paternu mogli bismo kreirati zasebne familije objekata za svaku ulogu, omogućavajući lakše dodavanje novih uloga u budućnosti (npr. premijum korisnik) bez mijenjanja postojeće strukture koda.

5. Builder pattern

Builder patern koristimo kada nam je potrebno primijeniti različite postupke za konstrukciju istog objekta.

U našoj aplikaciji, Builder patern bi se mogao primijeniti u procesu registracije korisnika. Klasa **Korisnik** sadrži više atributa (ime, prezime, email, šifra, uloga, datum registracije) čija se inicijalizacija može razlikovati ovisno o načinu registracije. Korisnik koji se prvi put registruje mogao bi odabrati između dva načina: automatskog generiranja korisničkog imena i lozinke putem **AutomatskiBuilder**-a, ili ručnog unosa tih podataka putem **RucniBuilder**-a. Oba buildera implementirali bi zajednički interfejs **Builder**, a klasa **KorisnikDirector** bi orkestrirala proces izgradnje bez ovisnosti o konkretnoj implementaciji.



PATERNI PONASANJA

State Pattern [implementacija]

State patern omogućava objektu da mijenja svoje ponašanje u zavisnosti od stanja u kojem se trenutno nalazi. Ovaj patern je pogodan za sisteme u kojima objekti prolaze kroz više različitih stanja, pri čemu svako stanje definiše različita pravila ponašanja i dozvoljene operacije.

U sistemu za prodaju i iznajmljivanje računarske opreme ovaj patern može biti primijenjen na upravljanje rezervacijama i transakcijama. Rezervacija može imati statuse *Na čekanju*, *Potvrđena*, *Otkazana* i *Završena*, dok transakcija može imati statuse *Na čekanju*, *Završena* i *Otkazana*. U zavisnosti od trenutnog statusa dozvoljene su različite akcije. Na primjer, rezervacija koja je već potvrđena ne može ponovo biti potvrđena, dok se otkazana rezervacija ne može završiti.

Implementacijom State paterna svako stanje bi bilo predstavljeno zasebnom klasom koja definiše dozvoljene operacije i prelaze u druga stanja. Na taj način se izbjegava veliki broj uslovnih naredbi, a sistem postaje jednostavniji za održavanje i proširivanje.

Observer Pattern [implementacija]

Observer patern uspostavlja vezu između objekata tako da promjena stanja jednog objekta automatski obavještava sve objekte koji su zainteresovani za tu promjenu. Ovaj patern je posebno pogodan za sisteme koji koriste različite vrste obavještenja.

U našem sistemu osnovni način komunikacije sa korisnicima predstavlja elektronska pošta. Observer patern može biti iskorišten za slanje automatskih obavještenja prilikom promjene statusa rezervacije ili transakcije. Na primjer, kada prodavac potvrdi rezervaciju artikla, kupac automatski dobija obavještenje putem e-maila. Slično tome, korisnik može biti obaviješten o otkazivanju rezervacije, završetku transakcije ili promjeni statusa zahtjeva za uklanjanje artikla.

Implementacijom Observer paterna postiže se odvajanje logike poslovnih procesa od logike slanja obavještenja, čime sistem postaje pregledniji i lakši za održavanje.

Strategy Pattern

Strategy pattern koristi se kada za isti problem postoji više različitih algoritama ili načina izvršavanja određene funkcionalnosti, pri čemu korisnik ili sistem može odabrati odgovarajući algoritam u toku rada. Na taj način se pojedinačni algoritmi izdvajaju u zasebne klase, dok ostatak sistema ostaje nezavisan od njihove implementacije.

U trenutnoj verziji sistema za prodaju i iznajmljivanje računarske opreme ovaj pattern nije neophodan, ali bi mogao biti vrlo koristan u budućim nadogradnjama. Jedna od mogućih primjena bila bi implementacija različitih načina pretrage artikala. Korisnicima bi se mogla omogućiti pretraga po nazivu, kategoriji, cijeni, proizvođaču ili prosječnoj ocjeni proizvoda, pri čemu bi svaki način pretrage predstavljao zasebnu strategiju.

Također, Strategy pattern bi se mogao primijeniti prilikom obračuna cijene iznajmljivanja. Sistem bi u budućnosti mogao podržavati dnevni, sedmični i mjesečni najam, kao i različite modele obračuna za fizička i pravna lica. U tom slučaju bi svaki način obračuna bio predstavljen posebnom strategijom, dok bi ostatak sistema koristio jedinstven interfejs bez potrebe za izmjenama postojećeg koda.

Template Method Pattern

Template Method pattern koristi se kada više procesa ima istu osnovnu strukturu, ali se pojedini koraci razlikuju u zavisnosti od konkretne situacije. Osnovni algoritam definira se u nadklasi, dok se specifični koraci implementiraju u izvedenim klasama.

U budućem razvoju sistema ovaj pattern mogao bi biti iskorišten za obradu različitih tipova transakcija. Proces kupovine i proces iznajmljivanja imaju veliki broj zajedničkih koraka, kao što su provjera korisnika, provjera dostupnosti artikla, kreiranje transakcije i slanje obavještenja korisnicima. Međutim, određeni dijelovi procesa razlikuju se u zavisnosti od vrste transakcije.

Na primjer, kod iznajmljivanja potrebno je evidentirati period trajanja najma i rezervirati artikal za određeni vremenski period, dok se kod kupovine nakon završetka transakcije artikal trajno uklanja iz ponude. Primjenom Template Method patterna zajednička struktura procesa ostala bi ista, dok bi se specifični koraci implementirali u posebnim klasama za kupovinu i iznajmljivanje.

Iterator Pattern

Iterator pattern omogućava prolazak kroz kolekciju objekata bez potrebe da korisnik poznaje način na koji su podaci organizovani i pohranjeni. Na taj način se postiže jednostavniji pristup podacima i veća fleksibilnost sistema.

U budućim verzijama sistema ovaj pattern mogao bi se koristiti za pregled velikog broja artikala, transakcija, rezervacija i korisnika. Kako broj podataka raste, javlja se potreba za efikasnim prolaskom kroz različite kolekcije i generisanjem izvještaja.

Jedna od mogućih primjena bila bi prikaz najbolje ocijenjenih artikala. Prilikom dodavanja novih ocjena sistem bi mogao koristiti Iterator pattern za prolazak kroz sve aktivne artikale i izračunavanje novih rang-lista. Slično tome, administrator bi mogao koristiti iterator za generisanje statističkih izvještaja o prodaji, iznajmljivanju ili aktivnosti korisnika bez poznavanja interne strukture podataka.

Command Pattern

Command pattern pretvara zahtjev za izvršavanje određene akcije u zaseban objekt koji sadrži sve informacije potrebne za izvršenje te akcije. Na taj način omogućava se evidentiranje izvršenih operacija, njihovo odgađanje ili eventualno poništavanje.

U trenutnoj verziji sistema ovaj pattern nije neophodan, ali bi mogao biti koristan u budućem razvoju administratorskog i moderatorskog dijela aplikacije. Akcije kao što su deaktivacija korisnika, odobravanje zahtjeva za uklanjanje artikla ili brisanje neprimjerenog sadržaja mogle bi biti predstavljene zasebnim komandama.

Na taj način sistem bi mogao voditi evidenciju svih izvršenih akcija, što bi bilo korisno za reviziju rada administratora i moderatora. Također, u budućnosti bi bilo moguće implementirati funkcionalnosti poput poništavanja određenih akcija ili izvršavanja više akcija u unaprijed definisanom redoslijedu.

Memento dizajn pattern

Memento dizajn patern omogućava čuvanje prethodnog stanja objekta bez narušavanja njegove enkapsulacije, te vraćanje objekta u ranije stanje kada je to potrebno. Ovaj patern je posebno koristan u sistemima gdje korisnici često vrše izmjene podataka i postoji potreba za poništavanjem ili vraćanjem prethodnih verzija.

U trenutnoj verziji sistema za prodaju i iznajmljivanje računarske opreme primjena ovog paternna nije neophodna, ali bi mogao biti veoma koristan u budućim nadogradnjama. Na primjer, prodavač može uređivati informacije o artiklu kao što su naziv, opis, cijena, slike ili dostupnost proizvoda. Prije svake izmjene sistem bi mogao sačuvati prethodno stanje artikla u obliku Memento objekta. U slučaju greške ili neželjene izmjene, prodavač ili administrator mogli bi vratiti artikal na prethodnu verziju bez potrebe za ručnim unosom podataka. Slična primjena moguća je i kod korisničkih profila, gdje bi se mogla čuvati istorija izmjena podataka korisnika, kao i kod administrativnih akcija koje utiču na sadržaj sistema. Na taj način povećala bi se sigurnost podataka i olakšalo upravljanje izmjenama unutar sistema.

Visitor dizajn pattern

Visitor dizajn patern omogućava dodavanje novih operacija nad postojećim objektima bez potrebe za izmjenom njihovih klasa. Ovaj patern je naročito pogodan kada postoji stabilna struktura objekata nad kojima se vremenom pojavljuje potreba za izvršavanjem novih funkcionalnosti.

U budućem razvoju sistema Visitor patern mogao bi biti iskorišten za generisanje različitih statističkih izvještaja i analiza poslovanja. Sistem sadrži više vrsta objekata kao što su korisnici, artikli, rezervacije i transakcije, a nad njima je moguće izvršavati različite analize.

Na primjer, jedan Visitor mogao bi izračunavati ukupan broj realizovanih prodaja, drugi broj aktivnih iznajmljivanja, treći prosječnu ocjenu artikala, dok bi četvrti generisao izvještaj o najaktivnijim prodavačima. Umjesto dodavanja posebnih metoda za svaku novu analizu u postojeće klase, svaka nova funkcionalnost bila bi implementirana kao zaseban Visitor. Ovakav pristup omogućava jednostavnije proširivanje sistema u budućnosti, posebno ukoliko se ukaže potreba za naprednim izvještavanjem, analizom poslovanja ili generisanjem statistika za administratore sistema. Na taj način nove funkcionalnosti mogu biti dodane bez izmjena postojećih klasa, čime se poštuje Open/Closed princip objektno orijentisanog dizajna.